

Data Structure:

연결 자료구조와 연결 리스트

Prof. Jungmin Kwon
Spring, 2026

■ 학습목표

- 연결 자료구조를 이해한다.
- 순차 자료구조와 연결 자료구조의 차이점과 장단점을 알아본다.
- 연결 리스트의 종류와 특징을 알아본다.
- 연결 자료구조를 이용한 다항식의 덧셈 연산 방법을 알아본다.

■ 내용

- 01 연결 자료구조와 연결 리스트의 이해
- 02 단순 연결 리스트
- 03 원형 연결 리스트
- 04 이중 연결 리스트
- 05 연결 리스트의 응용 및 구현

1. 연결 자료구조와 연결 리스트의 이해

■ 순차 자료구조의 문제점

- 삽입 연산이나 삭제 연산 후에 연속적인 물리 주소를 유지하기 위해서 원소들을 이동시키는 추가 작업과 시간 소요
 - 원소들의 이동 작업으로 인한 오버헤드로 원소의 개수가 많고 삽입 · 삭제 연산이 많이 발생하는 경우에 성능상의 문제 발생

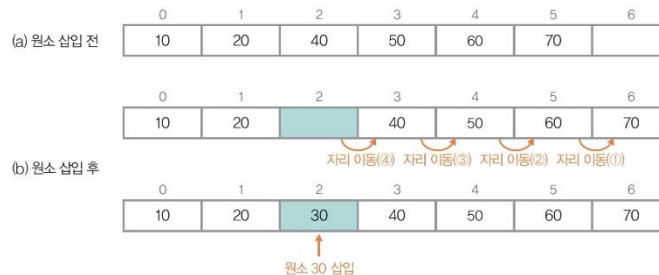


그림 3-11 선형 리스트에서의 원소 삽입

- 순차 자료구조는 배열을 이용해 구현하기 때문에 배열이 갖고 있는 메모리 사용의 비효율성 문제를 그대로 가짐
- 순차 자료구조에서의 연산 시간에 대한 문제와 저장 공간에 대한 문제를 개선한 자료 표현 방법 필요

1. 연결 자료구조와 연결 리스트의 이해

■ 연결 자료구조 Linked Data Structure

– 자료의 논리적인 순서와 물리적인 순서가 불일치

- 각 원소에 저장되어 있는 다음 원소의 주소에 의해 순서가 연결되는 방식
 - ▶ **물리적인 순서**를 맞추기 위한 오버헤드가 발생하지 않음

```
int sale[4] = { 157, 209, 251, 312 };
```

(a) 분기별 판매량 선형 리스트의 1차원 배열 선언

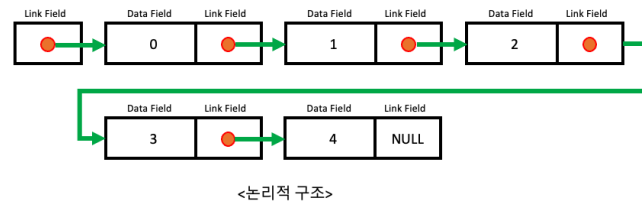
	[0]	[1]	[2]	[3]
sale	157	209	251	312

(b) 분기별 판매량 선형 리스트의 논리적 구조

그림 3-5 분기별 판매량 선형 리스트 예

메모리 주소		
α	157	[0]
$\alpha + 4\text{바이트}$	209	[1]
$\alpha + (2 \times 4\text{바이트})$	257	[2]
$\alpha + (3 \times 4\text{바이트})$	312	[3]

(c) 분기별 판매량 선형 리스트의 물리적 구조



start	100	Link Field
100번지	0	Data Field
	160	Link Field
120번지	2	Data Field
	320	Link Field
160번지	1	Data Field
	120	Link Field
300번지	4	Data Field
	NULL	Link Field
320번지	3	Data Field
	300	Link Field

<물리적 구조>

- 여러 개의 작은 공간을 연결하여 하나의 전체 자료구조를 표현
 - ▶ 크기 변경이 유연하고 더 효율적으로 메모리를 사용

1. 연결 자료구조와 연결 리스트의 이해

■ 연결 자료구조 Linked Data Structure

– 연결 리스트

- 데이터를 리스트의 연결 자료구조로 표현
- 연결하는 방식에 따라
 - 단순 연결 리스트
 - 원형 연결 리스트
 - 이중 연결 리스트
 - 이중 원형 연결 리스트



(a) 이불과 퀼트 이불



(b) 소시지와 줄줄이 소시지

그림 4-1 순차 자료구조와 연결 자료구조 예

1. 연결 자료구조와 연결 리스트의 이해

■ 연결 리스트의 노드

- 노드란, 연결 자료구조에서 하나의 원소를 표현하기 위한 단위 구조
- <원소, 주소>의 구조



그림 4-2 노드의 논리적 구조

- 데이터 필드 data field
 - 원소의 값을 저장
 - 저장할 원소의 형태에 따라서 하나 이상의 필드로 구성
- 링크 필드 link field
 - 다음 노드의 주소를 저장
 - 포인터 변수를 사용하여 주소 값을 저장

1. 연결 자료구조와 연결 리스트의 이해

■ 순차 자료구조와 연결 자료구조의 비교

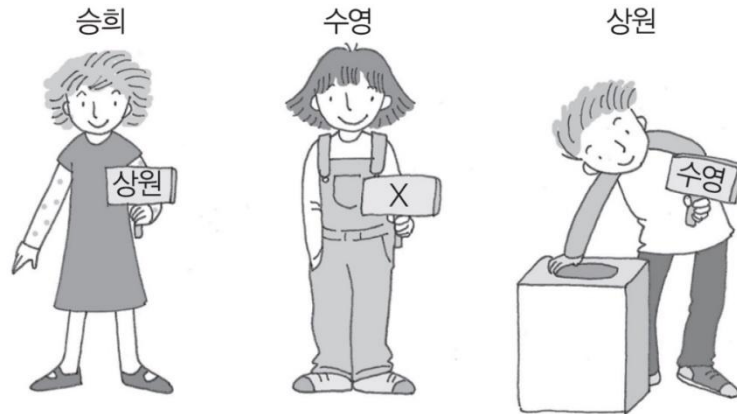
표 4-1 순차 자료구조와 연결 자료구조의 비교

구분	순차 자료구조	연결 자료구조
메모리 저장 방식	필요한 전체 메모리 크기를 계산하여 할당하고, 할당된 메모리의 시작 위치부터 빈자리 없이 자료를 순서대로 연속하여 저장한다.	노드 단위로 메모리가 할당되며, 저장 위치의 순서와 상관없이 노드의 링크 필드에 다음 자료의 주소를 저장한다.
연산 특징	삽입·삭제 연산 후에도 빈자리 없이 자료가 순서대로 연속 저장되어, 변경된 논리적인 순서와 저장된 물리적인 순서가 일치한다.	삽입·삭제 연산 후 논리적인 순서가 변경되어도 링크 정보만 변경되고 물리적 위치는 변경되지 않는다.
프로그램 기법	배열을 이용한 구현	포인터를 이용한 구현

1. 연결 자료구조와 연결 리스트의 이해

- 연결 리스트의 이해
 - 기차놀이

① 이름표 뽑기



② 뽑은 사람을 찾아 연결 : 승희 → 상원 → 수영



그림 4-3 기차놀이 : 이름표를 뽑아 이름표대로 기차 연결하기

1. 연결 자료구조와 연결 리스트의 이해

- 연결 리스트의 이해
 - 기차놀이와 연결 리스트
 - 기차놀이 하는 아이들 : 연결 리스트의 노드
 - 이름표 : 노드의 link 필드



그림 4-4 기차놀이의 노드 표현

1. 연결 자료구조와 연결 리스트의 이해

- 선형 리스트 week의 순차 리스트 표현
 - 리스트 week=(월, 화, 수, 목, 금, 토, 일)



(a) 논리적 구조

그림 4-5 선형 리스트 week의 순차 리스트 표현



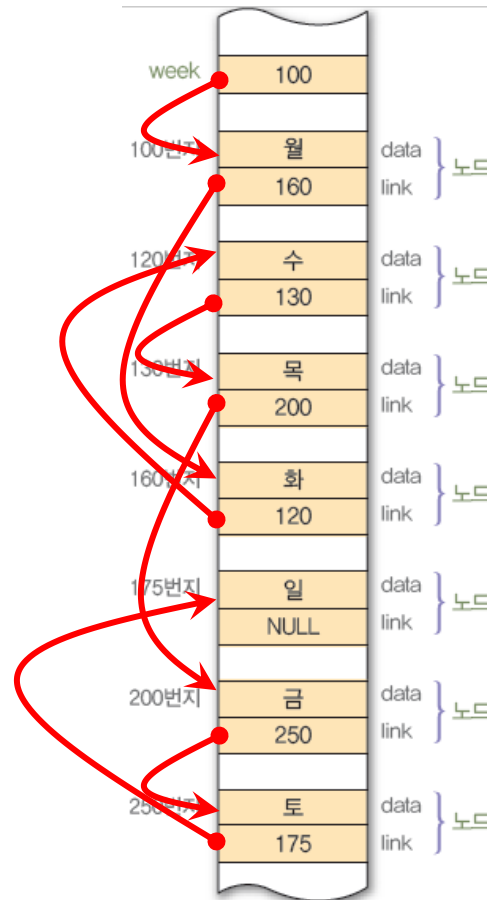
(b) 물리적 구조

1. 연결 자료구조와 연결 리스트의 이해

- 선형 리스트 week의 연결 리스트 표현



(a) 논리적 구조

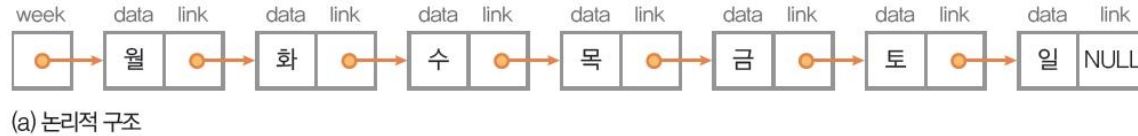


(b) 물리적 구조

그림 4-6 선형 리스트 week의 연결 리스트 표현

1. 연결 자료구조와 연결 리스트의 이해

– 선형 리스트와 연결 리스트의 비교



- **리스트 이름** week : 연결 리스트의 시작을 가리키는 포인터 변수
 - week는 연결 리스트의 첫 번째 노드를 가리킴과 동시에 연결된 리스트 전체 의미
- 연결 리스트의 마지막 노드의 링크 필드 : 노드 끝을 표시하기 위해 **NULL(널)**을 저장



그림 4-7 공백 연결 리스트

1. 연결 자료구조와 연결 리스트의 이해

– 선형 리스트와 연결 리스트의 비교

- **공백 연결 리스트** : 포인터 변수 week에 NULL 저장(널 포인터)



그림 4-7 공백 연결 리스트

- 각 노드의 필드에 저장한 값은 **점 연산자**를 사용해 액세스
 - week.data : 포인터 week가 가리키는 노드 데이터 필드 값 '월'
 - week.link : 포인터 week가 가리키는 노드 링크 필드에 저장된 주소값 '120'

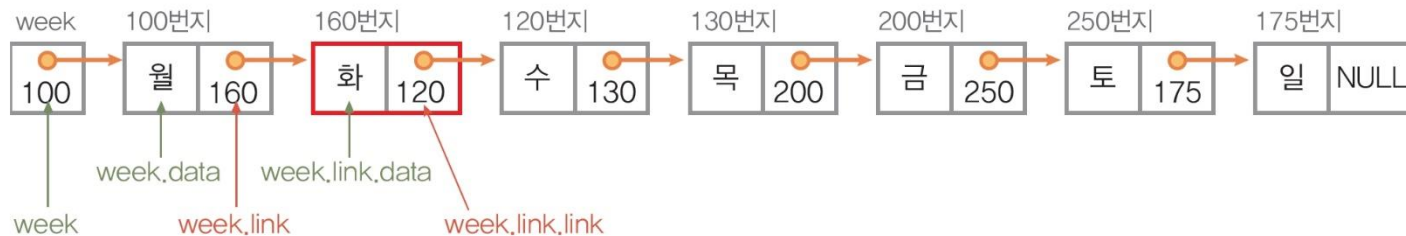


그림 4-8 선형 리스트 week의 연결 리스트 구조

1. 연결 자료구조와 연결 리스트의 이해

– 리스트 week의 노드에 대한 구조체 정의

```
typedef struct Node {
    char data[4];
    struct Node* link;
};
```

char data[4] 는
저장하려는 자료형에 따라 변화한다.

저장하려는 자료에 맞게 선언한다.

그림 4-9 리스트 week 노드의 구조체

- 구조체의 data 값에 접근 할 때는 구조체 이름 뒤에 점 . 을 찍음
 - Ex) 구조체 week.data or week.link

1. 연결 자료구조와 연결 리스트의 이해

- 연결 리스트의 노드와 점 연산자 표현

- (a)의 대화에 담긴 진호와 상원이의 관계를
- (b)의 연결 리스트의 노드로 표현하고,
- (c)와 같이 점 연산자(.)를 사용하여 액세스

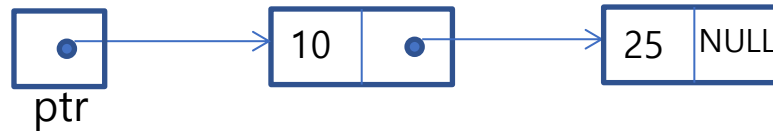


(c) 진호.link.link.link.link.link.link.link.data

그림 4-10 연결 리스트의 노드와 점 연산자 표현

2. 단순 연결 리스트

- 단순 연결 리스트 singly linked list의 개념
 - 노드가 하나의 링크 필드에 의해서 다음 노드와 연결되는 구조를 가짐
 - 연결 리스트
 - 선형 연결 리스트 linear linked list
 - 단순 연결 선형 리스트 singly linked linear list



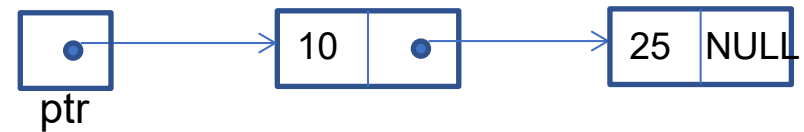
2. 단순 연결 리스트

- 연결 리스트를 C 로 코딩하기
 - malloc() 를 사용.
 - 노드 구조체 선언



```
typedef struct Node {
    int data;
    struct Node *link;
} Node;
```

< 연결리스트 구성 >



```
Node *ptr, *p, *q;
p = (Node*) malloc( sizeof(Node) );
p->data = 25;
p->link = NULL;
```

```
q = (Node*) malloc( sizeof(Node) );
q->data = 10;
q->link = p;
```

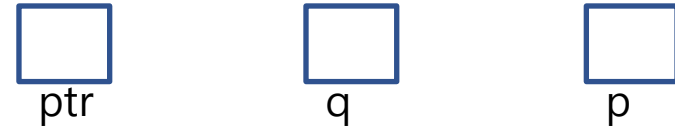
```
ptr = q;
```

2. 단순 연결 리스트

■ 연결 리스트를 C 로 코딩하기 (단계별 변화)

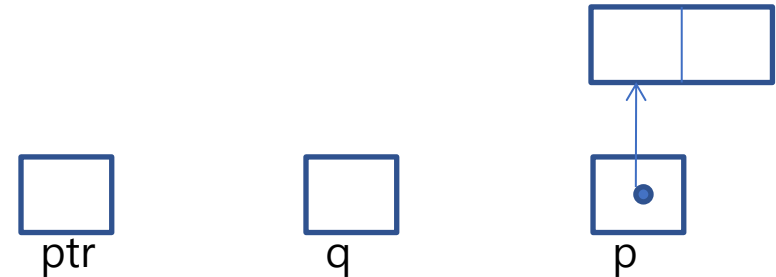
1. 포인터 변수 선언

```
Node *ptr, *p, *q;
```



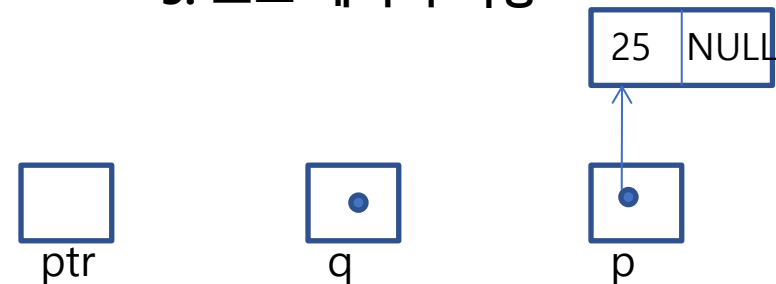
2. 노드 메모리공간 확보

```
Node *ptr, *p, *q;  
p = (Node*) malloc( sizeof(Node) );
```



3. 노드 데이터 저장

```
Node *ptr, *p, *q;  
p = (Node*) malloc( sizeof(Node) );  
p->data = 25;  
p->link = NULL;
```

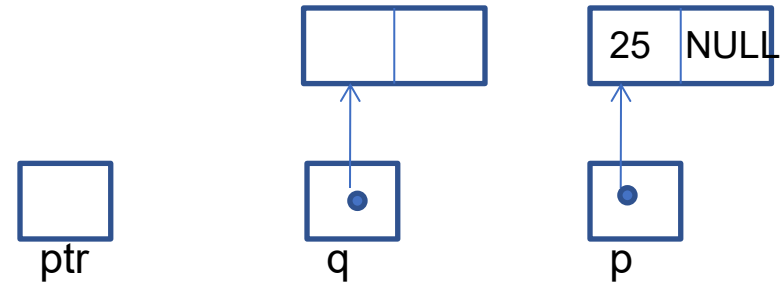


2. 단순 연결 리스트

■ 연결 리스트를 C 로 코딩하기 (단계별 변화)

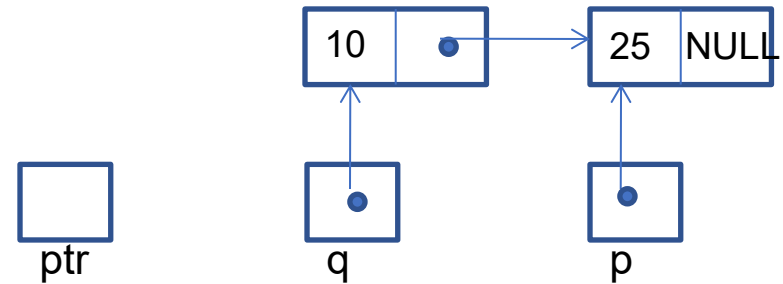
```
Node *ptr, *p, *q;
p = (Node*) malloc( sizeof(Node) );
p->data = 25;
p->link = NULL;
```

```
q = (Node*) malloc( sizeof(Node) );
```



```
Node *ptr, *p, *q;
p = (Node*) malloc( sizeof(Node) );
p->data = 25;
p->link = NULL;
```

```
q = (Node*) malloc( sizeof(Node) );
q->data = 10;
q->link = p;
```



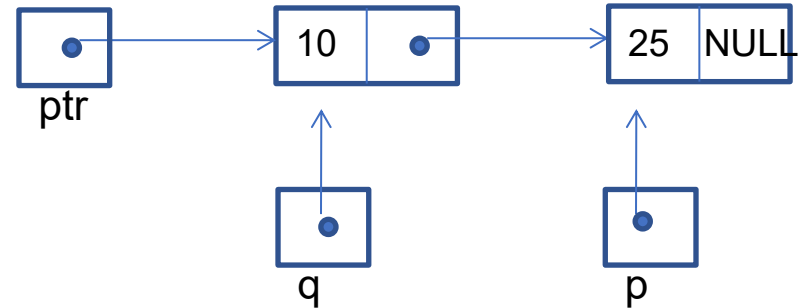
2. 단순 연결 리스트

■ 연결 리스트를 C 로 코딩하기 (단계별 변화)

```
Node *ptr, *p, *q;
p = (Node*) malloc( sizeof(Node) );
p->data = 25;
p->link = NULL;
```

```
q = (Node*) malloc( sizeof(Node) );
q->data = 10;
q->link = p;
```

```
ptr = q;
```



2. 단순 연결 리스트 : 삽입 연산

■ 단순 연결 리스트에서의 삽입 연산

0 기차놀이에 철이를 끼우기 전 상태



1 앞사람에게 이름표 받아 오기



2 앞사람에게 자기 이름표 주기



3 이름표대로 연결하기



2. 단순 연결 리스트 : 삽입 연산

■ 단순 연결 리스트에 삽입하는 방법

- ① 삽입할 노드를 준비한다.
- ② 새 노드의 데이터 필드에 값을 저장한다.
- ③ 새 노드의 링크값을 지정한다.
- ④ 리스트의 앞 노드에 새 노드를 연결한다.

그림 4-11 단순 연결 리스트에 노드를 삽입하는 방법

2. 단순 연결 리스트 : 삽입 연산

- 단순 연결 리스트 week2=(월, 금, 일)에서 ‘월’과 ‘금’ 사이에 ‘수’ 삽입 과정
 - 초기상태

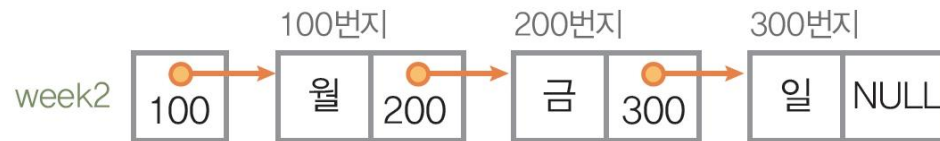
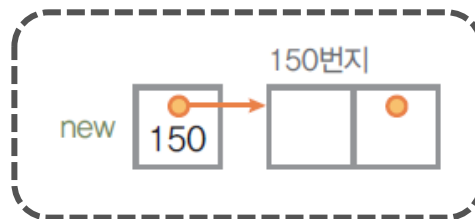
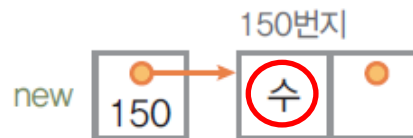


그림 4-12 단순 연결 리스트 week2에 노드를 삽입하기 전인 초기 상태

- ① 삽입할 노드 준비 : 공백 노드를 가져와 포인터 변수 new가 가리키게 함

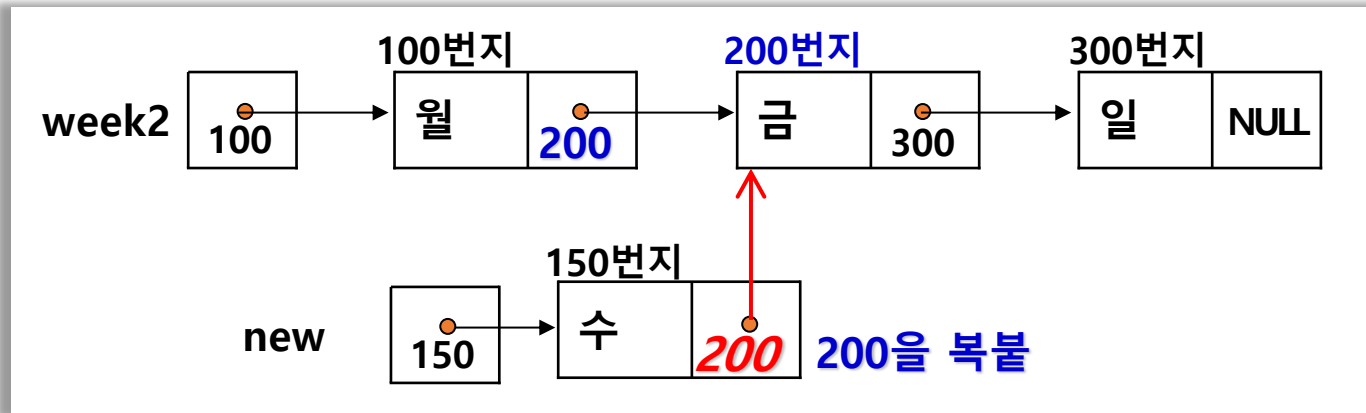


- ② 새 노드의 데이터 필드값 저장 : new의 데이터 필드에 ‘수’를 저장

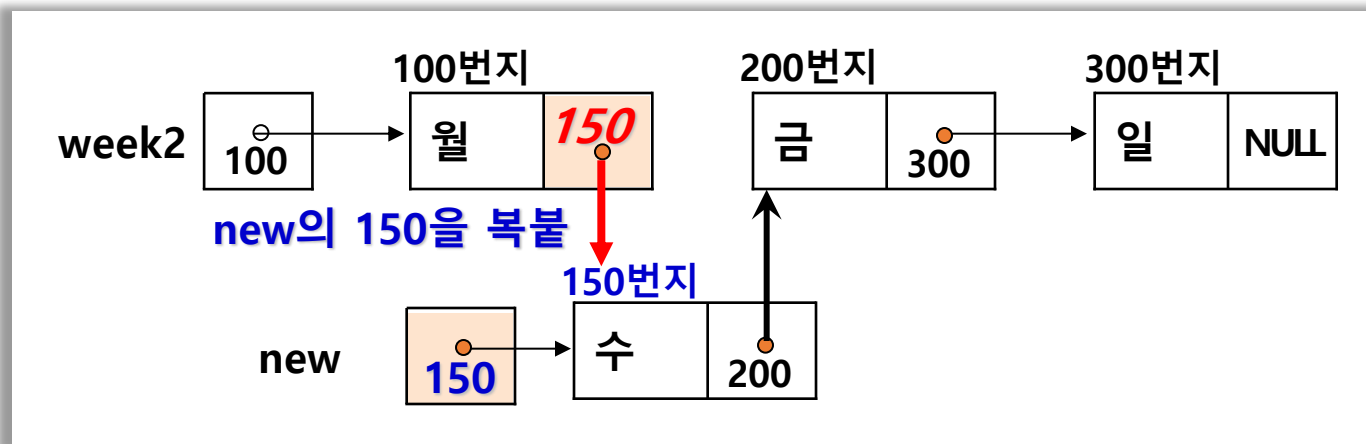


2. 단순 연결 리스트 : 삽입 연산

- ③ 새 노드의 링크 필드값 지정 : new의 앞 노드, 즉 '월' 노드의 링크 필드 값을 new 노드의 링크 필드에 저장



- ④ 리스트의 앞 노드에 새 노드 연결 : new의 값을 '월' 노드의 링크 필드에 저장



2. 단순 연결 리스트 : 삭제 연산

■ 단순 연결 리스트에서의 삭제 연산

0 기차놀이에서 철이가 빠지기 전 상태



1 앞사람에게 이름표 넘겨주기



2 이름표대로 연결하기



2. 단순 연결 리스트 : 삭제 연산

- 단순 연결 리스트에서 노드를 삭제하는 방법

- ① 삭제할 노드의 앞 노드를 찾는다.
- ② 앞 노드에 삭제할 노드의 링크 필드값을 저장한다.
- ③ 삭제한 노드의 앞 노드와 삭제한 노드의 다음 노드를 연결한다.

그림 4-13 단순 연결 리스트에서 노드를 삭제하는 방법

2. 단순 연결 리스트 : 삭제 연산

- 단순 연결 리스트 week2=(월, 수, 금, 일)에서 원소 '수' 삭제 과정
 - 초기 상태



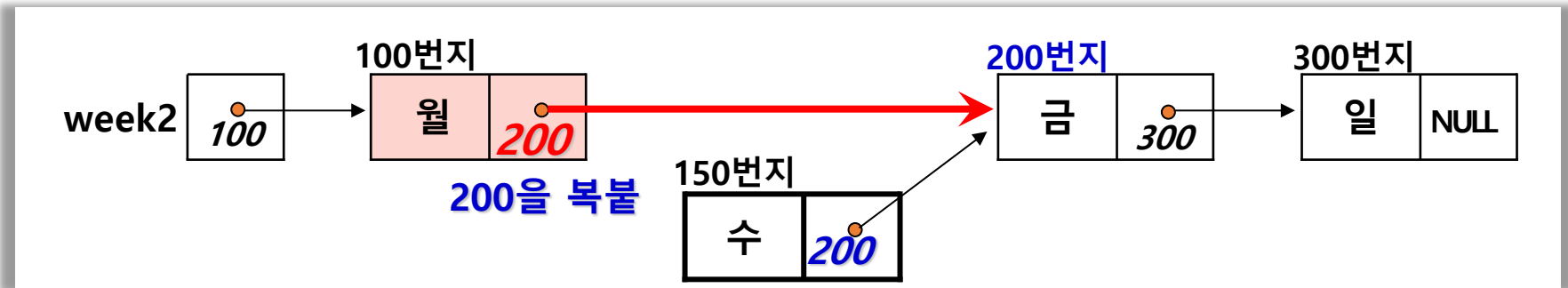
그림 4-14 단순 연결 리스트 week2에서 '수' 노드를 삭제하기 전의 초기 상태

- 1 앞 노드를 찾음 : 삭제할 원소의 앞 노드(선행자)를 찾음



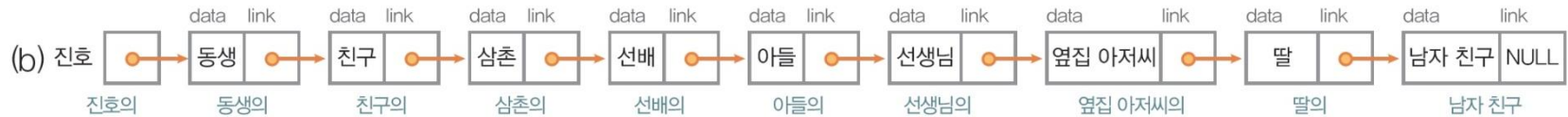
2. 단순 연결 리스트 : 삭제 연산

- ② 앞 노드에 삭제할 노드의 링크 필드값 저장 : 삭제할 원소 '수'의 링크 필드 값을 앞 노드의 링크 필드에 저장
- ③ 삭제한 노드의 앞뒤 노드 연결 : 삭제한 노드의 앞 노드인 '월' 노드를 삭제한 노드의 다음 노드인 '금' 노드에 연결



2. 단순 연결 리스트

- 단순 연결 리스트의 알고리즘
 - 첫 번째 노드로 삽입하기
 - 중간 노드로 삽입하기
 - 마지막 노드로 삽입하기



2. 단순 연결 리스트

- 단순 연결 리스트의 알고리즘
 1. 첫 번째 노드로 삽입하기

알고리즘 4-1 단순 연결 리스트의 첫 번째 노드 삽입

```

insertFirstNode(L, x)
  ① new ← getNode();
  ② new.data ← x;
  ③ new.link ← L;
  ④ L ← new;
end insertFirstNode()

```

L : 리스트의 시작 포인터
 x : 삽입되는 데이터

2. 단순 연결 리스트

- 1 새로운 노드 생성: $\text{new} \leftarrow \text{getNode}();$

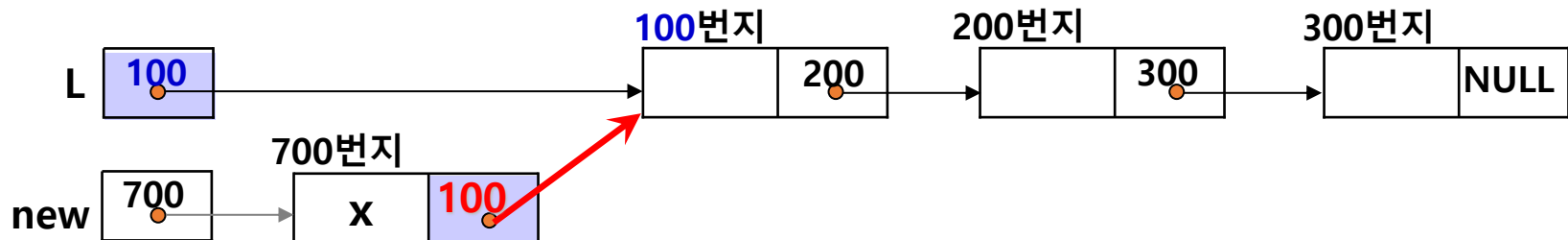


- 2 데이터 필드 값 저장: $\text{new.data} \leftarrow x;$



- 3 new에 link 필드 값 저장: $\text{new.link} \leftarrow L;$

삽입할 노드를 연결하기 위해서 **리스트의 첫 번째 노드 주소(L)**를 삽입할 새 노드 **new의 링크 필드(new.link)**에 저장하여, 새 노드 new가 리스트의 첫 번째 노드를 가리키게 한다.



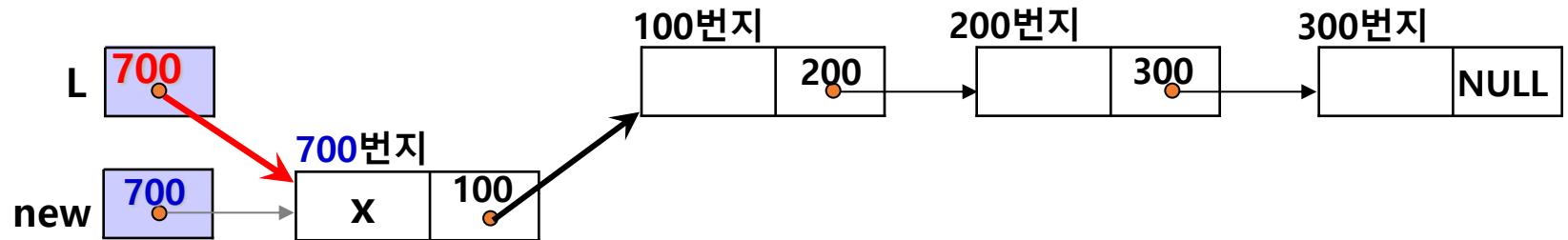
알고리즘 4-1 단순 연결 리스트의 첫

```
insertFirstNode(L, x)
1 new ← getNode();
2 new.data ← x;
3 new.link ← L;
4 L ← new;
end insertFirstNode()
```

2. 단순 연결 리스트

- ④ L이 new를 가리키도록 새로운 값 저장: $L \leftarrow new$;

리스트의 첫 번째 노드 주소를 저장하고 있는 **포인터 L에**, **새 노드의 주소 new를** 저장하여, 포인터 L이 새 노드를 첫 번째 노드로 가리키도록 지정



알고리즘 4-1 단순 연결 리스트의 첫

```
insertFirstNode(L, x)
  ① new ← getNode();
  ② new.data ← x;
  ③ new.link ← L;
  ④ L ← new;
end insertFirstNode()
```


2. 단순 연결 리스트

2. 중간 노드로 삽입하기

- 리스트 L에서 포인터변수 pre가 가리키고 있는 노드의 다음에 데이터 필드 값이 x인 새 노드를 삽입하는 알고리즘

알고리즘 4-2 단순 연결 리스트의 중간 노드 삽입

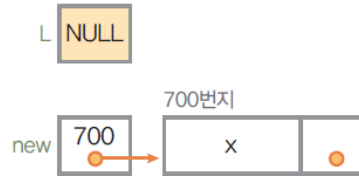
```

insertMiddleNode(L, pre, x)
    ① new ← getNode();
    ② new.data ← x;
    {
        ③ if (L = NULL) then {
            ③-a L ← new;
            ③-b new.link ← NULL;
        }
        {
            ④ else {
                ④-a new.link ← pre.link;
                ④-b pre.link ← new;
            }
        }
    }
end insertMiddleNode()

```

2. 단순 연결 리스트

- ① $new \leftarrow getNode();$
- ② $new.data \leftarrow x;$
- ③ 경우 1: L이 공백 리스트인 경우



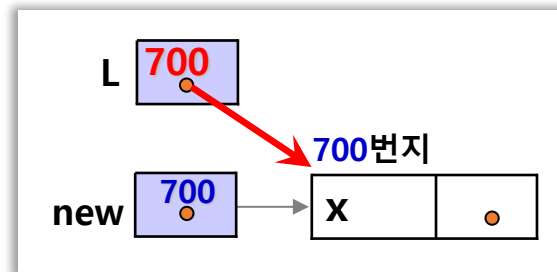
알고리즘 4-2 단순 연결 리스트의 중간 노드 삽입

```

insertMiddleNode(L, pre, x)
1 new ← getNode();
2 new.data ← x;
3 if (L = NULL) then {
4     ③-① L ← new;
5     ③-② new.link ← NULL;
6 }
7 else {
8     ④-① new.link ← pre.link;
9     ④-② pre.link ← new;
10 }
end insertMiddleNode()
    
```

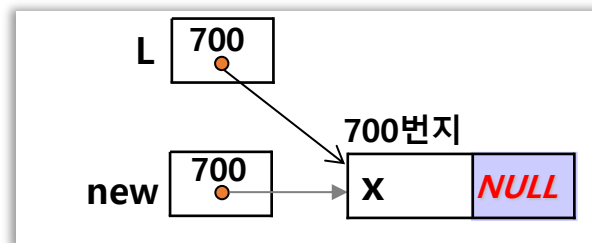
- ③- a $L \leftarrow new;$

리스트 포인터 L에 새 노드 new의 주소를 저장하여, 새 노드 new가 리스트의 첫 번째 노드가 되도록 함

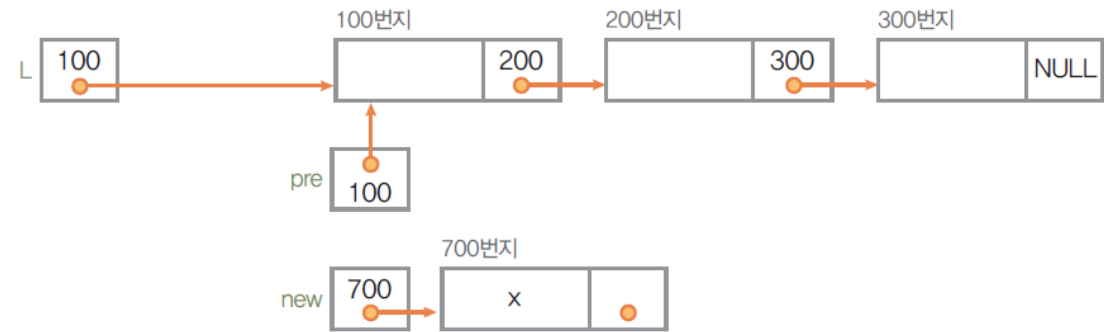


- ③- b $new.link \leftarrow NULL;$

리스트의 마지막 노드인 new의 링크 필드에 NULL을 저장해 마지막 노드임을 표시



④ 경우 2: 공백 리스트가 아닌 경우

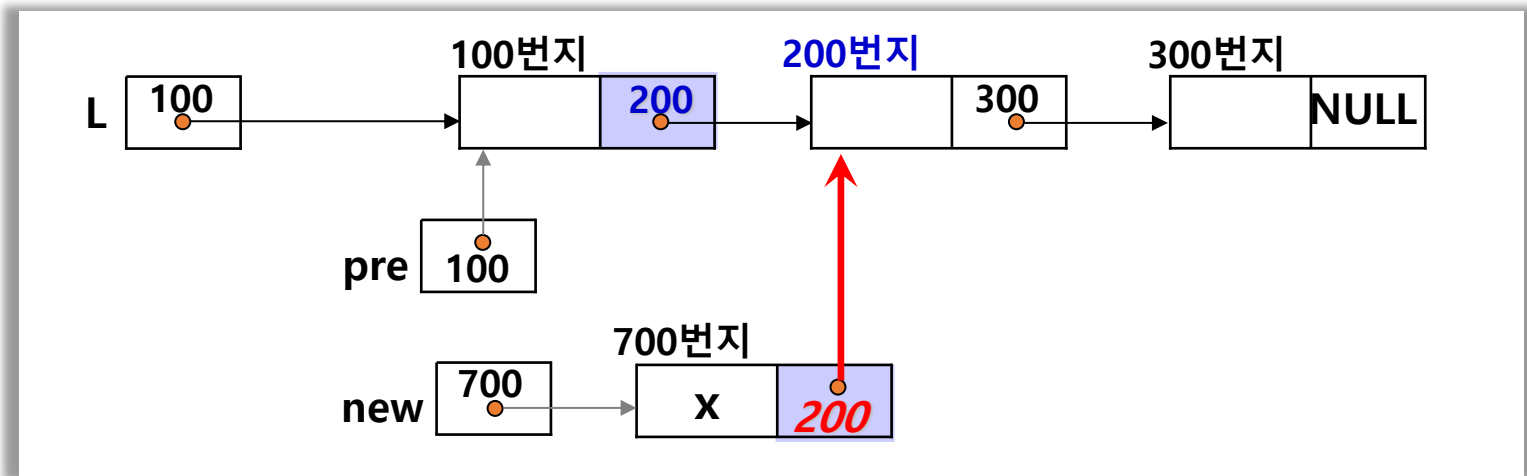


알고리즘 4-2 단순 연결 리스트의 중간 노드 삽입

```
insertMiddleNode(L, pre, x)
1 new ← getNode();
2 new.data ← x;
3 if (L = NULL) then {
4   L ← new;
5   new.link ← NULL;
6 }
7 else {
8   new.link ← pre.link;
9   pre.link ← new;
10 }
end insertMiddleNode()
```

④- a new.link ← pre.link;

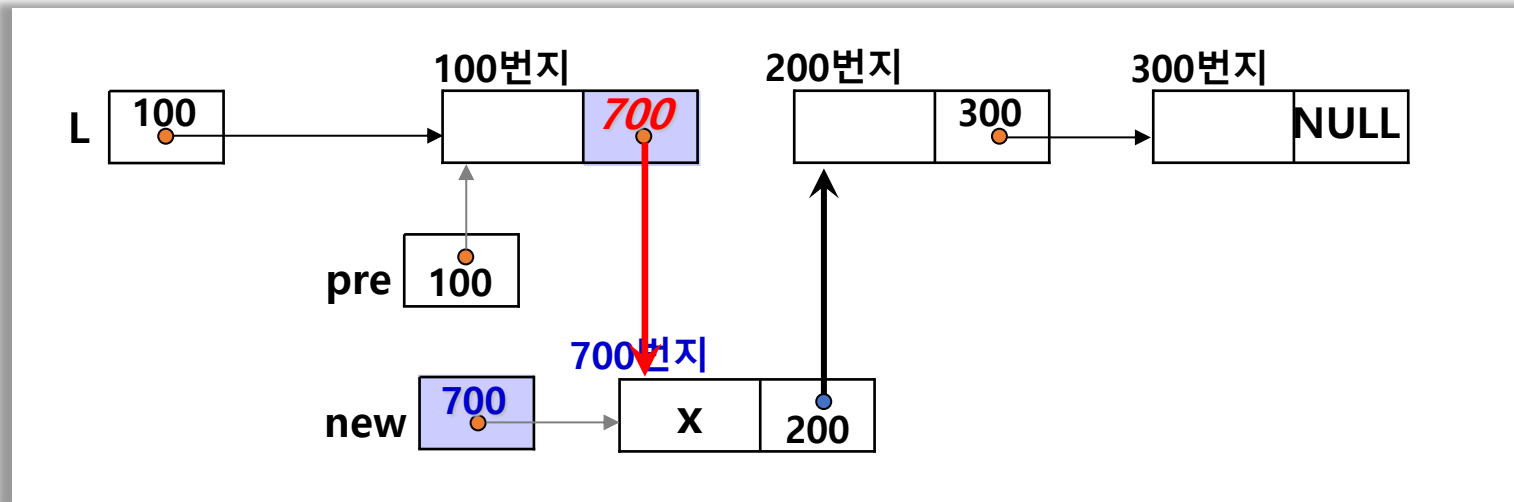
노드 pre의 링크 필드 값을 노드 new의 링크 필드에 저장하여, 새 노드 new가 노드 pre의 다음 노드를 가리키도록 함



2. 단순 연결 리스트

④- b $\text{pre.link} \leftarrow \text{new};$

포인터 **new**의 값을 노드 **pre**의 링크 필드에 저장하여, 노드 **pre**가 새 노드 **new**를 다음 노드로 가리키도록 함



알고리즘 4-2 단순 연결 리스트의 중간 노드 삽입

```
insertMiddleNode(L, pre, x)
1 new ← getNode();
2 new.data ← x;
3 if (L = NULL) then {
4   ① L ← new;
5   ② new.link ← NULL;
6 }
7 else {
8   ③ new.link ← pre.link;
9   ④ pre.link ← new;
10 }
end insertMiddleNode()
```

2. 단순 연결 리스트

3. 마지막 노드로 삽입하기

- 새 노드 new를 마지막 노드로 삽입하는 알고리즘

알고리즘 4-3 단순 연결 리스트의 마지막 노드 삽입

```

insertLastNode(L, x)
    ① new ← getNode();
    ② new.data ← x;
    ③ new.link ← NULL;
    ④ { if (L == NULL) then {
        ④-a L ← new;
        return;
    }
    ⑤ { ⑤-a temp ← L;
        while (temp.link ≠ NULL) do
            ⑤-b temp ← temp.link;
        ⑤-c temp.link ← new;
    }
end insertLastNode()

```

2. 단순 연결 리스트

알고리즘 4-3 단순 연결 리스트의 마지막 노드 삽입

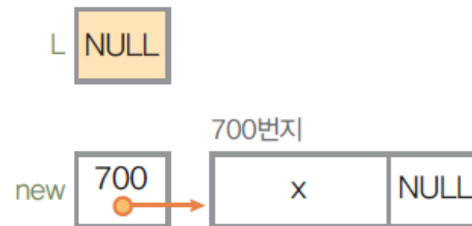
```
insertLastNode(L, x)
1 new ← getNode();
2 new.data ← x;
3 new.link ← NULL;
4 if (L == NULL) then {
    4-1 L ← new;
    return;
}
5-1 temp ← L;
while (temp.link ≠ NULL) do
    5-2 temp ← temp.link;
5-3 temp.link ← new;
end insertLastNode()
```

① new ← getNode();

② new.data ← x;

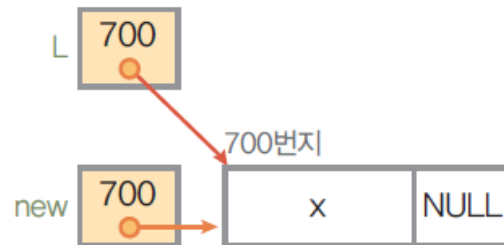
③ new.link ← NULL;

④ 경우 1: 공백 리스트인 경우



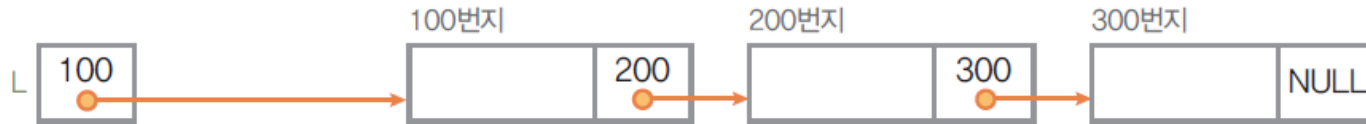
④-a L ← new;

리스트 포인터 L에 새 노드 new의 주소(700) 저장. new는 리스트 L의 첫 번째 노드이자 마지막 노드



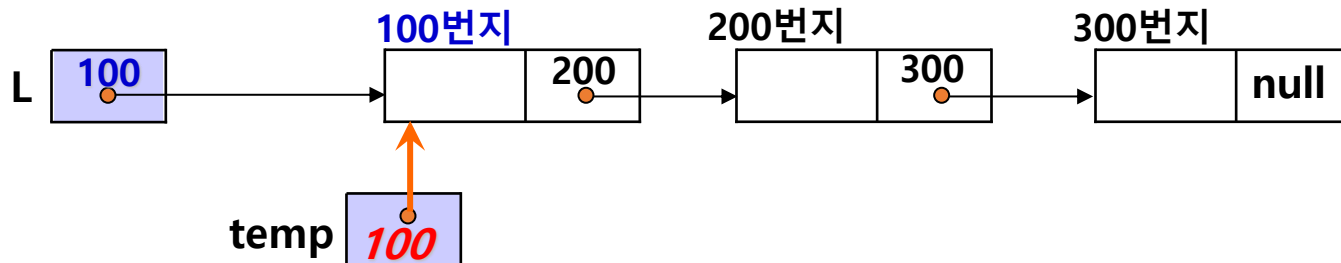
2. 단순 연결 리스트

⑤ 경우 2: 공백 리스트가 아닌 경우



⑤- a temp ← L;

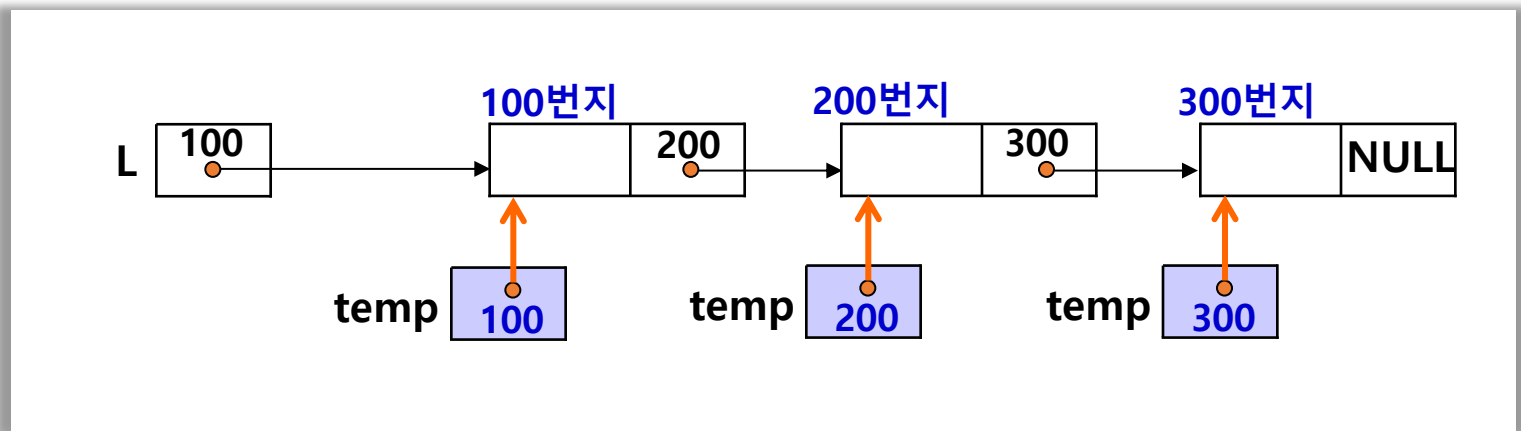
현재 리스트 L의 마지막 노드를 찾기 위해서 노드를 **순회할 임시포인터 temp**에 리스트의 **첫 번째 노드의 주소(L)**를 지정



2. 단순 연결 리스트

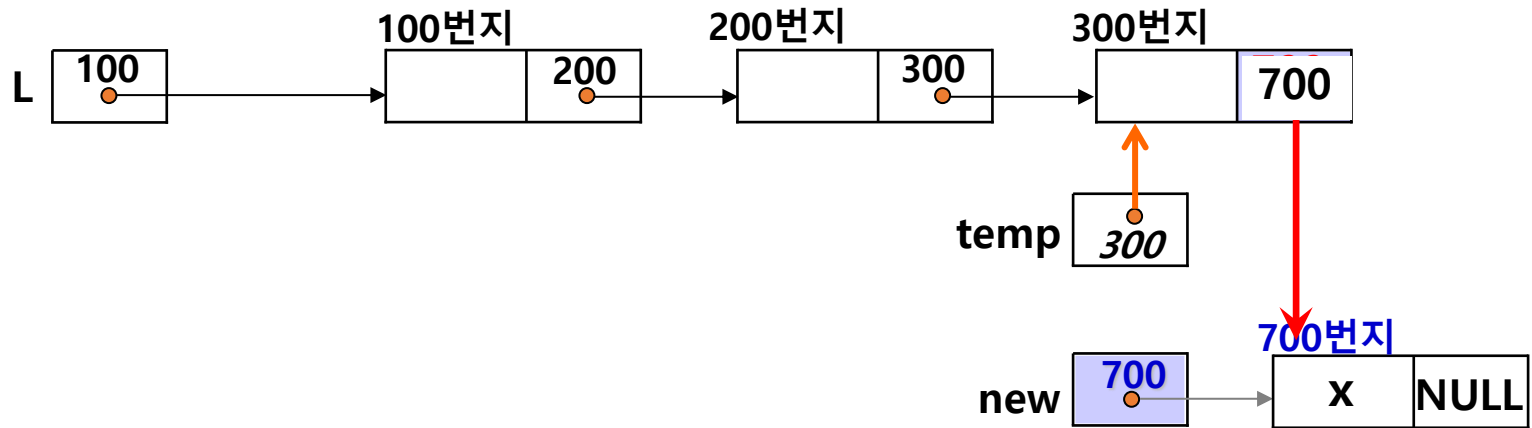
⑤- b temp $\leftarrow$ temp.link;

- 순회 포인터 temp가 가리키는 노드의 링크 필드가 NULL이 아닌 동안 (while (temp.link $\neq$ NULL)) 링크 필드를 따라 이동.
- 링크 필드가 NULL인 노드 즉, 마지막 노드를 찾으면 while 문을 끝내고 ⑤- c를 수행



2. 단순 연결 리스트

5- c temp.link ← new;



2. 단순 연결 리스트

- [예제 4-1] 단순 연결 리스트 프로그램 1 : [교재 174p](#)
- 실행 결과

(1) 공백 리스트 생성하기!

$L = ()$

(2) 리스트에 [수] 노드 삽입하기!

$L = (\text{수})$

(3) 리스트 마지막에 [금] 노드 삽입하기!

$L = (\text{수}, \text{금})$

(4) 리스트 첫 번째에 [월] 노드 삽입하기!

$L = (\text{월}, \text{수}, \text{금})$

(5) 리스트 공간을 해제하여 공백 리스트로 만들기!

$L = ()$

- 구조:

2. 단순 연결 리스트

예제 4-1 단순 연결 리스트에 노드 삽입하기

[InsertLinkedList.h]

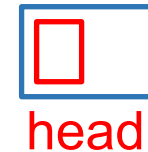
```

01  #pragma once
02  // 단순 연결 리스트의 노드 구조를 구조체로 정의
03  typedef struct ListNode {
04      char data[4];
05      struct ListNode* link;
06  } ListNode;
07
08  // 리스트 시작을 나타내는 head 노드를 구조체로 정의
09  typedef struct {
10      ListNode* head;
11  } linkedList_h;
12
13  linkedList_h* createLinkedList_h(void);
14  void freeLinkedList_h(linkedList_h* L);
15  void printList(linkedList_h * L);
16  void insertFirstNode(linkedList_h * L, char* x);
17  void insertMiddleNode(linkedList_h * L, ListNode * pre, char* x);
18  void insertLastNode(linkedList_h * L, char* x);
    
```

listNode



linkedList_h



[InsertLinkedList.c]

```

01  #define _CRT_SECURE_NO_WARNINGS
02  #include <string.h>
03  #include "InsertLinkedList.h"
04  // 공백 연결 리스트를 생성하는 연산
05  linkedList_h* createLinkedList_h(void) {
06      linkedList_h* L;
    
```

#include <stdlib.h>



2. 단순 연결 리스트

```
07     L = (linkedList_h*)malloc(sizeof(linkedList_h));
08     L -> head = NULL;    // 공백 리스트이므로 NULL로 설정
09     return L;
10 }
11
12 // 연결 리스트의 전체 메모리를 해제하는 연산
13 void freeLinkedList_h(linkedList_h* L) {
14     listNode* p;
15     while (L -> head != NULL) {
16         p = L -> head;
17         L -> head = L -> head -> link;
18         free(p);
19         p = NULL;
20     }
21 }
22
23 // 연결 리스트를 순서대로 출력하는 연산
24 void printList(linkedList_h* L) {
25     listNode* p;
26     printf("L = (");
27     p = L -> head;
28     while (p != NULL) {
29         printf("%s", p -> data);
30         p = p -> link;
31         if (p != NULL) printf(", ");
32     }
33     printf(") \n");
34 }
```

2. 단순 연결 리스트

```

35
36 // 첫 번째 노드로 삽입하는 연산 [알고리즘 4-1] 구현
37 void insertFirstNode(linkedList_h *L, char *x) {
38     listNode* newNode;
39     newNode = (listNode*)malloc(sizeof(listNode)); // 삽입할 새 노드 할당
40     strcpy(newNode->data, x); // 새 노드의 데이터 필드에 x 저장
41     newNode->link = L->head;
42     L->head = newNode;
43 }
44
45 // 노드를 pre 뒤에 삽입하는 연산 [알고리즘 4-2] 구현
46 void insertMiddleNode(linkedList_h* L, listNode *pre, char*x) {
47     listNode* newNode;
48     newNode = (listNode*)malloc(sizeof(listNode));
49     strcpy(newNode->data, x);
50     if (L->head == NULL) { // 공백 리스트인 경우
51         newNode->link = NULL; // 새 노드를 첫 번째이자 마지막 노드로 연결
52         L->head = newNode;
53     }

```

2. 단순 연결 리스트

```

54     else if (pre == NULL) {           // 삽입 위치를 지정하는 포인터 pre가 NULL인 경우
55         newNode -> link = L -> head   // 새 노드를 첫 번째 노드로 삽입
56         L -> head = newNode;
57     }
58     else {
59         newNode -> link = pre -> link; // 포인터 pre의 노드 뒤에 새 노드 연결
60         pre -> link = newNode;
61     }
62 }
63
64 // 마지막 노드로 삽입하는 연산 [알고리즘 4-3] 구현
65 void insertLastNode(linkedList_h* L, char* x) {
66     listNode* newNode;
67     listNode* temp;
68     newNode = (listNode*)malloc(sizeof(listNode));
69     strcpy(newNode -> data, x);
70     newNode -> link = NULL;
71     if (L -> head == NULL) {          // 현재 리스트가 공백인 경우
72         L -> head = newNode;          // 새 노드를 리스트의 시작 노드로 연결
73         return;
74     }
75     // 현재 리스트가 공백이 아닌 경우
76     temp = L -> head;
77     while (temp -> link != NULL) temp = temp -> link; // 현재 리스트의 마지막 노드를 찾음
78     temp -> link = newNode;          // 새 노드를 마지막 노드(temp)의 다음 노드로 연결
79 }

```

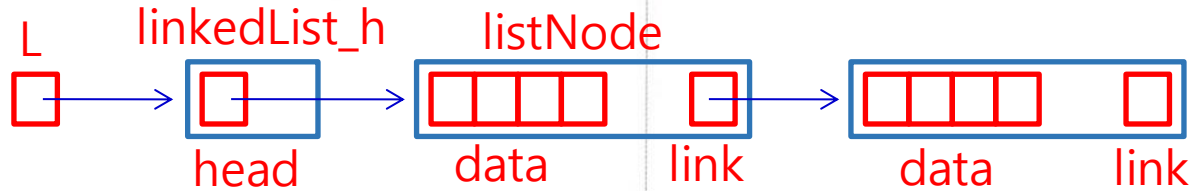

2. 단순 연결 리스트

[ex4_1.c]

```

01  #include <stdio.h>
02  #include "InsertLinkedList.h"
03
04  int main(void) {
05      linkedList_h* L;
06      L = createLinkedList_h();
07      printf("(1) 공백 리스트 생성하기! \n");
08      printList(L);
09
10      printf("\n(2) 리스트에 [수] 노드 삽입하기! \n");
11      insertFirstNode(L, "수");
12      printList(L);
13
14      printf("\n(3) 리스트 마지막에 [금] 노드 삽입하기! \n");
15      insertLastNode(L, "금");
16      printList(L);
17
18      printf("\n(4) 리스트 첫 번째에 [월] 노드 삽입하기! \n");
19      insertFirstNode(L, "월");
20      printList(L);
21
22      printf("\n(5) 리스트 공간을 해제하여 공백 리스트로 만들기! \n");
23      freeLinkedList_h(L);
24      printList(L);
25
26      getchar(); return 0;
27  }

```



2. 단순 연결 리스트

- 리스트 L에서 포인터 pre가 가리키는 노드의 다음 노드 삭제 알고리즘
 - 포인터 old: 삭제할 노드

알고리즘 4-4 단순 연결 리스트의 노드 삭제

```

deleteNode(L, pre)
  ① if (L == NULL) then error;
  {
    ②-a old ← pre.link;
    ②-b if (old == NULL) then return;
    ②-c pre.link ← old.link;
    ②-d returnNode(old);
  }
end deleteNode()
  
```


2. 단순 연결 리스트

- 삭제 연산 수행 과정

① 공백 리스트인 경우

공백 연결 리스트 L이 공백이면 오류 처리한다.

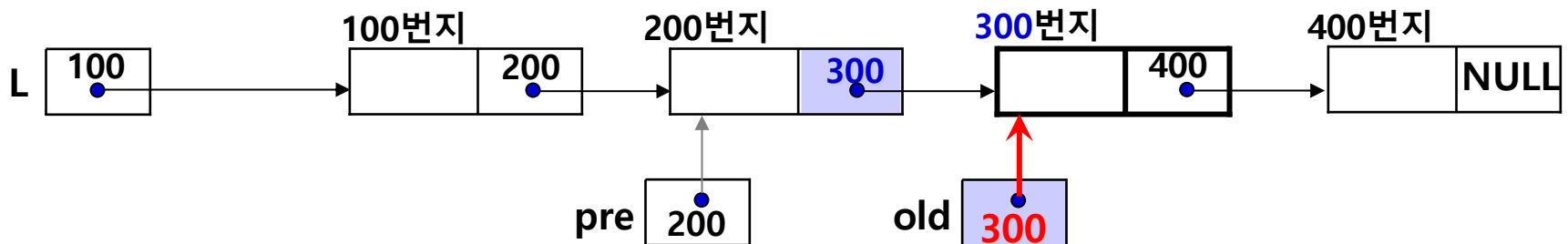
2. 단순 연결 리스트

- 삭제 연산 수행 과정

② 공백 리스트가 아닌 경우

②- a $\text{old} \leftarrow \text{pre.link};$

노드 **pre**의 다음노드의 주소(**pre.link**)를 **포인터 old**에 저장하여, 포인터 **old**가 다음 노드를 가리키게 한다.

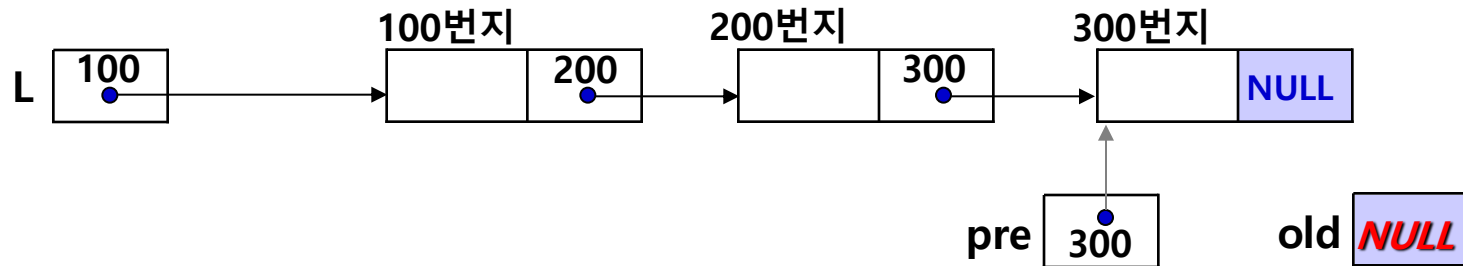


2. 단순 연결 리스트

②- b if (old = NULL) then return;

만약 노드 pre가 리스트의 마지막 노드였다면 :

포인터 pre가 가리키는 노드가 마지막 노드인 경우에,
 ②-① old $\leftarrow$ pre.link; 를 수행한 상태 :

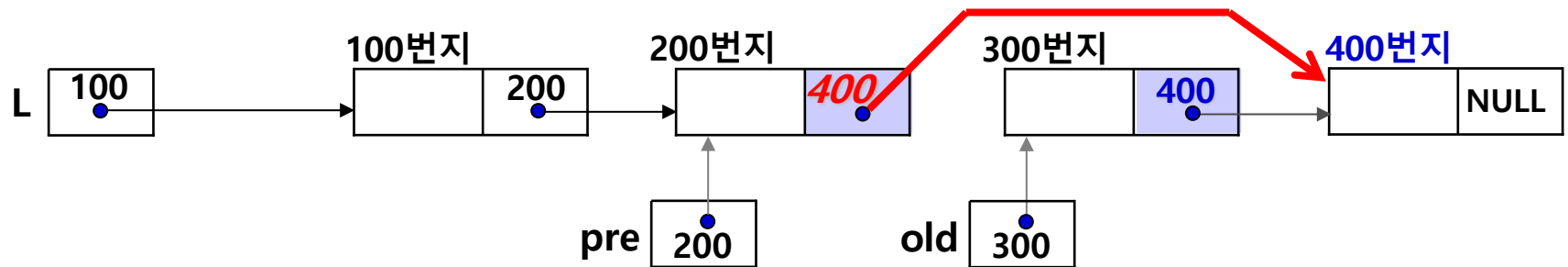


⇒ pre.link의 값은 null이므로 포인터 old의 값은 NULL이 된다.
 결국 노드 pre 다음에 삭제할 노드가 없다는 의미이므로 삭제연산을
 수행하지 못하고 반환(return)한다.

2. 단순 연결 리스트

②- c `pre.link ← old.link;`

삭제할 노드 `old`의 다음 노드(`old.link`)를 노드 `pre`의 다음 노드(`pre.link`)로 연결



②- d `returnNode(old);`

삭제한 노드 `old`의 메모리를 반환
리스트는 변경된 상태이다.

2. 단순 연결 리스트

- 노드를 탐색하는 알고리즘과 프로그램
 - 노드를 탐색하는 알고리즘
 - 리스트를 처음 노드부터 하나씩 순회하면서
 - 노드의 데이터 필드의 값과 x를 비교하여 일치하는 노드를 찾는 연산

알고리즘 4-5 단순 연결 리스트의 노드 탐색

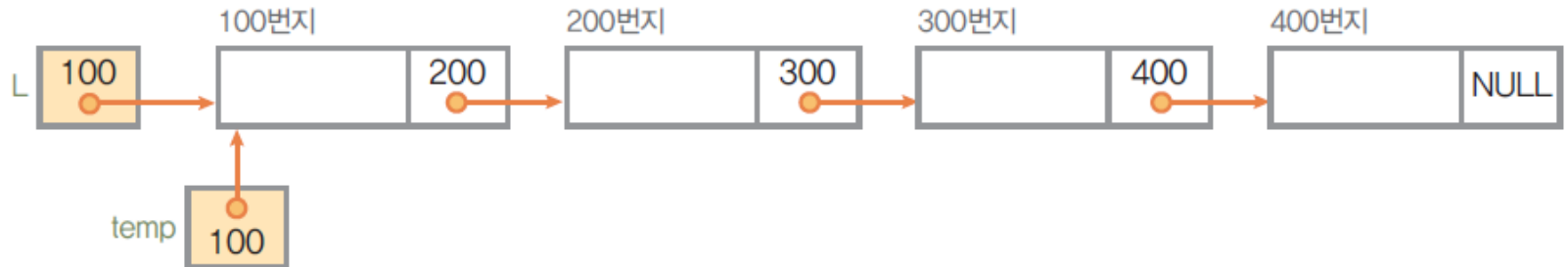
```

searchNode(L, x)
  ① temp ← L;
  ② { while (temp ≠ NULL) do {
      ②-a if (temp.data == x) then return temp;
      ②-b else temp ← temp.link;
    }
  ③ return temp;
end searchNode()
  
```

2. 단순 연결 리스트

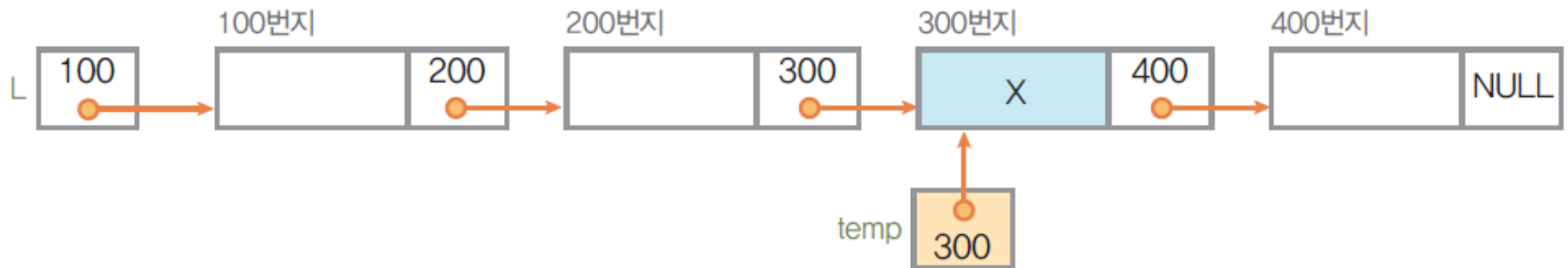
– 단순 연결 리스트에서 x 노드 탐색 과정

① $\text{temp} \leftarrow L;$



② 순회 포인터가 NULL이 아닌 경우

②- a if (temp.data = x) then return temp;



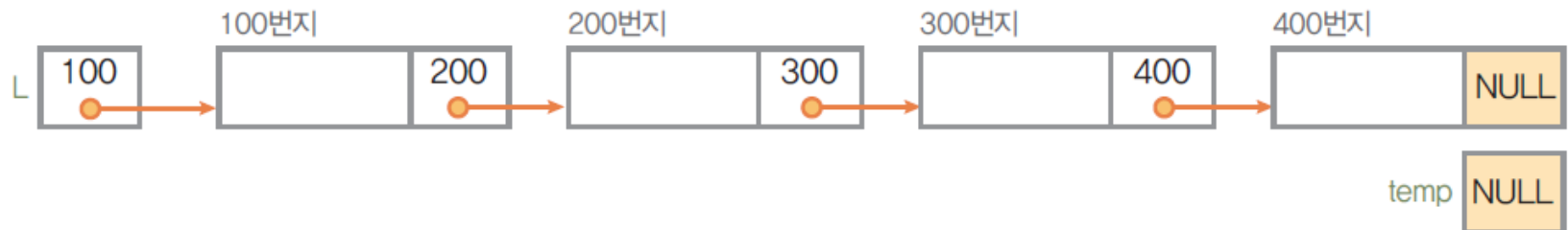
2. 단순 연결 리스트

– 단순 연결 리스트에서 x 노드 탐색 과정

②- b else temp $\leftarrow$ temp.link;

③ return temp;

while 문에서 x 를 찾지 못하고 끝나면 (temp 가 NULL 이 되면)
NULL 을 반환한다.



2. 단순 연결 리스트

- [예제 4-2] 단순 연결 리스트 프로그램 2 : [교재 180p](#)
- 실행 결과

(1) 리스트에 [월], [수], [일] 노드 삽입하기!
L = (월, 수, 일)

(2) 리스트에서 [수] 노드 탐색하기!
[수]를 찾았습니다.

(3) 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기!
L = (월, 수, 금, 일)

(4) 리스트에서 [일] 노드 삭제하기!
L = (월, 수, 금)

(5) 리스트의 순서를 역순으로 바꾸기!
L = (금, 수, 월)

2. 단순 연결 리스트

예제 4-2 단순 연결 리스트에 노드 삭제하고 탐색하기

[LinkedList.h]

```

001 #pragma once
002 // 단순 연결 리스트의 노드 구조를 구조체로 정의
003 typedef struct ListNode {
004     char data[4];
005     struct ListNode* link;
006 } listNode;
007
008 // 리스트의 시작을 나타내는 head 노드를 구조체로 정의
009 typedef struct {
010     listNode* head;
011 } linkedList_h;
012
013 linkedList_h* createLinkedList_h(void);
014 void freeLinkedList_h(linkedList_h* L);
015 void printList(linkedList_h* L);
016 void insertFirstNode(linkedList_h* L, char* x);
017 void insertMiddleNode(linkedList_h* L, listNode* pre, char* x);
018 void insertLastNode(linkedList_h* L, char* x);
019 void deleteNode(linkedList_h* L, listNode* p);
020 listNode* searchNode(linkedList_h* L, char* x);
021 void reverse(linkedList_h* L);

```

[LinkedList.c]

```

001 #define _CRT_SECURE_NO_WARNINGS
002 #include <stdio.h>
003 #include <stdlib.h>

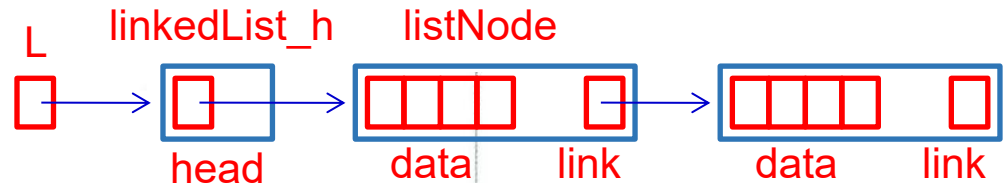
```

2. 단순 연결 리스트

```

004 #include <string.h>
005 #include "LinkedList.h"
006
... // 동일한 007~082행 생략 : [예제 4-1]의 [insertLinkedList.c]의 004~079행과 동일
083
084 // 리스트에서 노드 p를 삭제하는 연산
085 void deleteNode(linkedList_h* L, listNode* p) {
086     listNode* pre; // 삭제할 노드의 선행자 노드를 나타낼 포인터
087     if (L -> head == NULL) return; // 공백 리스트라면 삭제 연산 중단
088     if (L -> head -> link == NULL) { // 리스트에 노드가 한 개만 있는 경우
089         free(L -> head); // 첫 번째 노드를 메모리에서 해제하고
090         L -> head = NULL; // 리스트 시작 포인터를 NULL로 설정
091         return;
092     }
093     else if (p == NULL) return; // 삭제할 노드가 없다면 삭제 연산 중단
094     else { // 삭제할 노드 p의 선행자 노드를 포인터 pre를 이용해 찾음
095         pre = L -> head;
096         while (pre -> link != p) {
097             pre = pre -> link;
098         }
099         pre -> link = p -> link; // 삭제할 노드 p의 선행자 노드와 다음 노드를 연결
100         free(p); // 삭제 노드의 메모리 해제
101     }
102 }
103

```



2. 단순 연결 리스트

```

104 // 리스트에서 x 노드를 탐색하는 연산 [알고리즘 4-5] 구현
105 listNode* searchNode(linkedList_h* L, char* x) {
106     listNode *temp;
107     temp = L -> head;
108     while (temp != NULL) {
109         if (strcmp(temp -> data, x) == 0) return temp;
110         else temp = temp -> link;
111     }
112     return temp;
113 }

```

```

114
115 // 리스트의 노드 순서를 역순으로 바꾸는 연산 추가 연산
116 void reverse(linkedList_h * L) {
117     listNode* p;
118     listNode* q;
119     listNode* r;
120
121     p = L -> head;          // 포인터 p를 첫 번째 노드에 설정
122     q = NULL;
123     r = NULL;
124
125     // 리스트의 첫 번째 노드부터 링크를 따라 다음 노드로 이동하면서
126     // 노드 간의 연결을 바꿈
127     while (p != NULL) {
128         r = q;
129         q = p;
130         p = p -> link;
131         q -> link = r;
132     }
133     L -> head = q;
134 }

```

reverse() 동작 설명 – (교재참고)

2. 단순 연결 리스트

[ex4_2.c]

```

001  #include <stdio.h>
002  #include <stdlib.h>
003  #include "LinkedList.h"
004
005  int main(void) {
006      linkedList_h* L;
007      listNode *p;
008      L = createLinkedList_h(); // 공백 리스트 생성
009
010      printf("(1) 리스트에 [월], [수], [일] 노드 삽입하기! \n");
011      insertLastNode(L, "월"); insertLastNode(L, "수"); insertLastNode(L, "일");
012      printList(L);
013
014      printf("\n(2) 리스트에서 [수] 노드 탐색하기! \n");
015      p = searchNode(L, "수");
016      if (p == NULL) printf("찾는 데이터가 없습니다.\n");
017      else printf("[%s]를 찾았습니다.\n", p->data);
018
019      printf("\n(3) 리스트의 [수] 뒤에 [금] 노드 삽입하기! \n");
020      insertMiddleNode(L, p, "금");
021      printList(L);
022
023      printf("\n(4) 리스트에서 [일] 노드 삭제하기! \n");
024      p = searchNode(L, "일"); // 삭제할 노드 위치 p를 찾음
025      deleteNode(L, p); // 포인터 p 노드 삭제
026      printList(L);
027
028      printf("\n(5) 리스트 순서를 역순으로 바꾸기! \n");
029      reverse(L);
030      printList(L);
031
032      freeLinkedList_h(L); // 리스트의 메모리 해제
033      getchar();
034
035      return 0;
036  }

```

(1) 리스트에 [월], [수], [일] 노드 삽입하기!
L = (월, 수, 일)

(2) 리스트에서 [수] 노드 탐색하기!
[수]를 찾았습니다.

(3) 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기!
L = (월, 수, 금, 일)

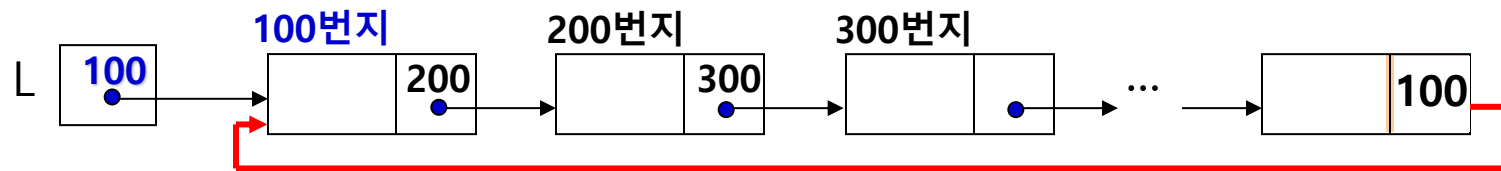
(4) 리스트에서 [일] 노드 삭제하기!
L = (월, 수, 금)

(5) 리스트의 순서를 역순으로 바꾸기!
L = (금, 수, 월)

3. 원형 연결 리스트

■ 원형 연결 리스트의 개념

- 단순 연결 리스트에서 마지막 노드가 리스트의 첫 번째 노드를 가리키게 하여 리스트의 구조를 **원형**으로 만든 연결 리스트
 - 단순 연결 리스트의 마지막 노드의 링크 필드에 첫 번째 노드의 주소를 저장하여 구성
 - 링크를 따라 계속 순회하면 이전 노드에 접근 가능
 - 예)



3. 원형 연결 리스트

- 선형 기차 놀이와 원형 기차놀이 비교



선형 기차 놀이



원형 기차 놀이



< 기차 놀이에 대한 노드 표현 >

4. 이중 연결 리스트

■ 이중 연결 리스트의 개념

- 양쪽 방향으로 순회할 수 있도록 노드를 연결한 리스트
- 이중 연결 리스트의 노드 구조와 구조체 정의
 - llink(left link) 필드 : 왼쪽 노드와 연결하는 포인터
 - rlink(right link) 필드 : 오른쪽 노드와 연결하는 포인터



(a) 노드 구조

```
typedef struct Dnode {
    struct Dnode *llink;
    char data[5];
    struct Dnode *rlink;
}
```

(b) 구조체 정의

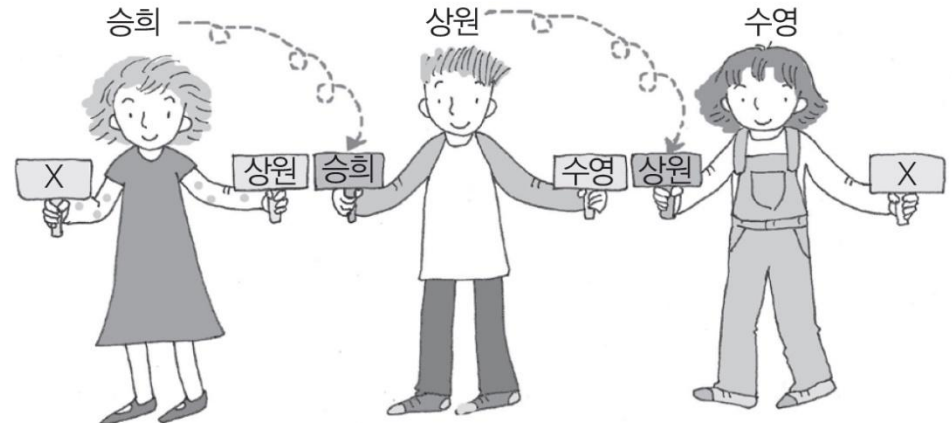
그림 4-26 이중 연결 리스트의 노드 구조와 구조체 정의

4. 이중 연결 리스트

- 단방향과 양방향 기차놀이



(a) 단방향 기차놀이



(b) 양방향 기차놀이

그림 4-27 단방향과 양방향 기차놀이

- (b)의 양방향 기차를 이중 연결 리스트라고 생각하면 아이들은 노드
- 왼손의 이름표는 llink, 오른손의 이름표는 rlink

4. 이중 연결 리스트

- 단순 연결 리스트 week2=(월, 수, 금, 일)에서 원소 '수' 삭제 과정

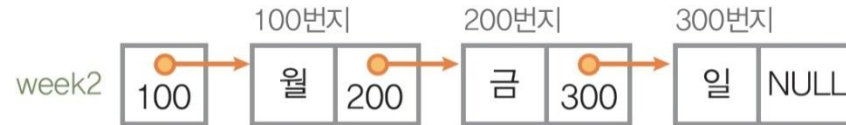


그림 4-12 단순 연결 리스트 week2에 노드를 삽입하기 전인 초기 상태

- 리스트 week=(월, 수, 금)의 이중 연결 리스트 구성



그림 4-28 리스트 week의 이중 연결 리스트

- 원형 이중 연결 리스트
 - 이중 연결 리스트를 원형으로 구성

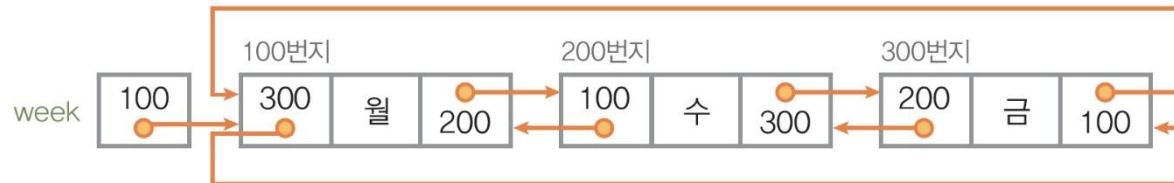
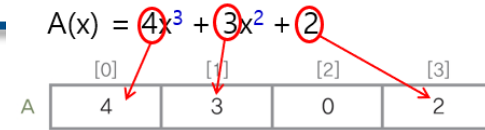


그림 4-29 리스트 week의 이중 원형 연결 리스트

5. 연결 리스트의 응용 및 구현 : 다항식 표현

■ 단순 연결 리스트를 이용한 다항식 표현

- 앞장에서는 순차리스트(배열)을 이용한 다항식을 다루었음.
- 다항식 노드의 노드 구조와 구조체 정의
- 노드 구조
 - 각 항에 대해서 계수와 지수를 저장
 - 계수를 저장하는 **coef**와 지수를 저장하는 **expo**의 두 개의 필드로 구성
 - 링크 필드 : 다음 항을 연결하는 포인터로 구성



(a) 노드 구조

```
typedef struct Node {
    float coef;
    int expo;
    struct Node *link;
};
```

(b) 구조체 정의

그림 4-36 다항식 노드의 노드 구조와 구조체 정의

5. 연결 리스트의 응용 및 구현 : 다항식 표현

- 다항식 노드를 사용한 표현

- 다항식 $A(x) = 4x^3 + 3x^2 + 5x$ 와 $B(x) = 3x^4 + x^3 + 2x + 1$

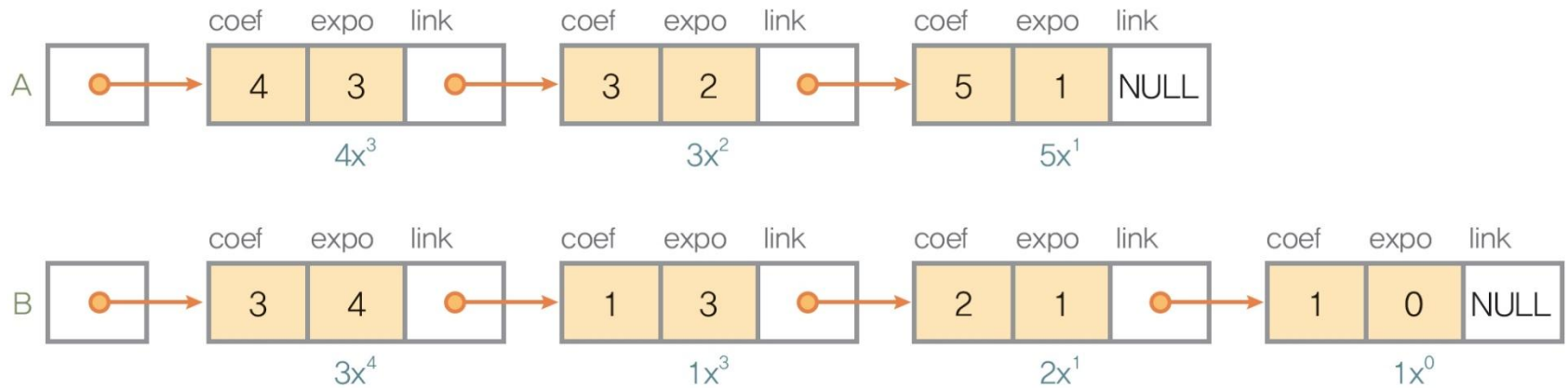


그림 4-37 다항식 A(x)와 B(x)의 단순 연결 리스트 표현

■ Review) 순차 리스트를 이용한 다항식 덧셈 연산

알고리즘 3-4 순차 리스트를 이용한 다항식의 덧셈 연산

```
addPoly(A, B)
// 주어진 두 다항식 A와 B를 더하여 결과 다항식 C를 반환하는 알고리즘
C ← zeroP();
while (not isZeroP(A) and not isZeroP(B)) do {
  case {
    maxExp(A) < maxExp(B) :
      C ← addTerm(C, coef(B, maxExp(B)), maxExp(B));

      B ← delTerm(B, maxExp(B));
    maxExp(A) == maxExp(B) :
      sum ← coef(A, maxExp(A)) + coef(B, maxExp(B));
      if (sum ≠ 0) then C ← addTerm(C, sum, maxExp(A));
      A ← delTerm(A, maxExp(A));
      B ← delTerm(B, maxExp(B));
    maxExp(A) > maxExp(B) :
      C ← addTerm(C, coef(A, maxExp(A)), maxExp(A));
      A ← delTerm(A, maxExp(A));
  }
}

if (not isZeroP(A)) then A의 나머지 항들을 C에 복사
else if (not isZeroP(B)) then B의 나머지 항들을 C에 복사
return C;

end addPoly()
```

5. 연결 리스트의 응용 및 구현 : 항 삽입

1. 다항식 연결 자료구조의 항 삽입 알고리즘

- 매개변수: 다항식 리스트 포인터 PL과 coef 필드 값을 저장한 변수 coef, expo 필드 값을 저장한 변수 expo, 리스트 PL의 마지막 노드의 위치를 지시하는 포인터 last

알고리즘 4-11 다항식 끝에 항 추가

```

appendTerm(PL, coef, expo, last)
    new ← getNode();
    new.expo ← expo;
    new.coef ← coef;
    new.link ← NULL;
    if (PL == NULL) then {
1 { 1-a PL ← new;
   1-b last ← new;
    }
    else {
2 { 2-a last.link ← new;
   2-b last ← new;
    }
end appendTerm()

```

5. 연결 리스트의 응용 및 구현 : 항 삽입

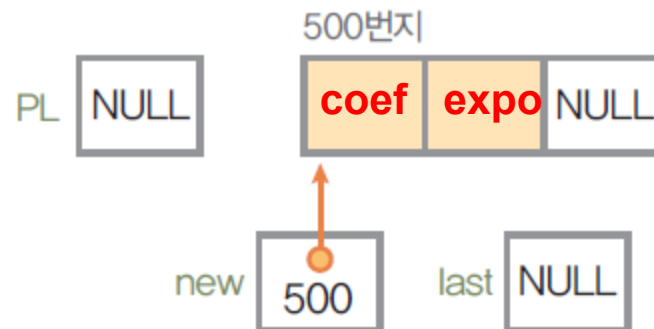
– 다항식에 항을 추가하는 과정

① 공백 리스트인 경우

1) 초기상태

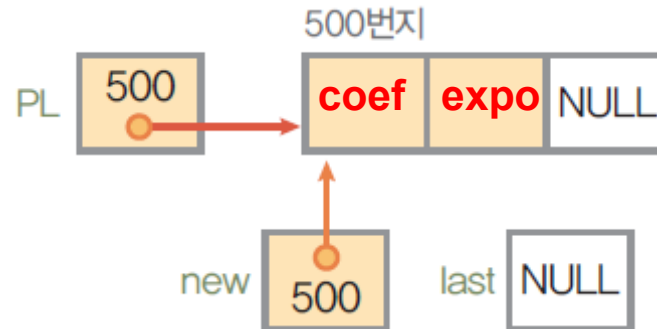


2) 노드를 만들고 노드 값을 저장

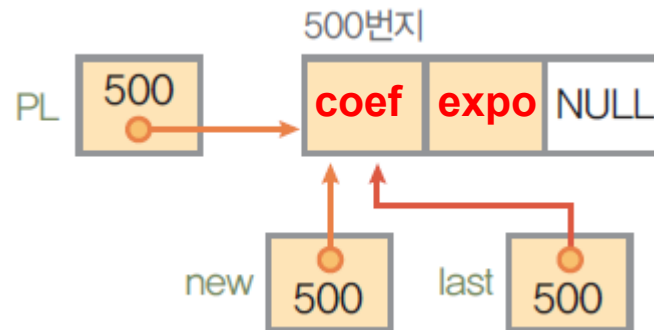


5. 연결 리스트의 응용 및 구현 : 항 삽입

①- a $PL \leftarrow new;$



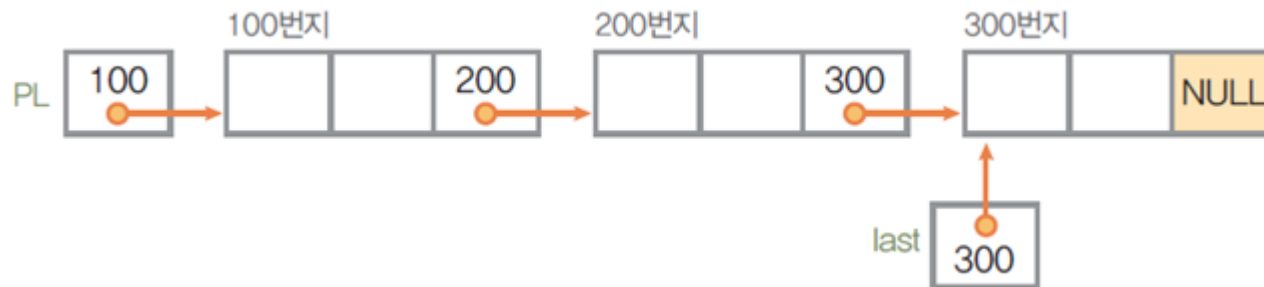
①- b $last \leftarrow new;$



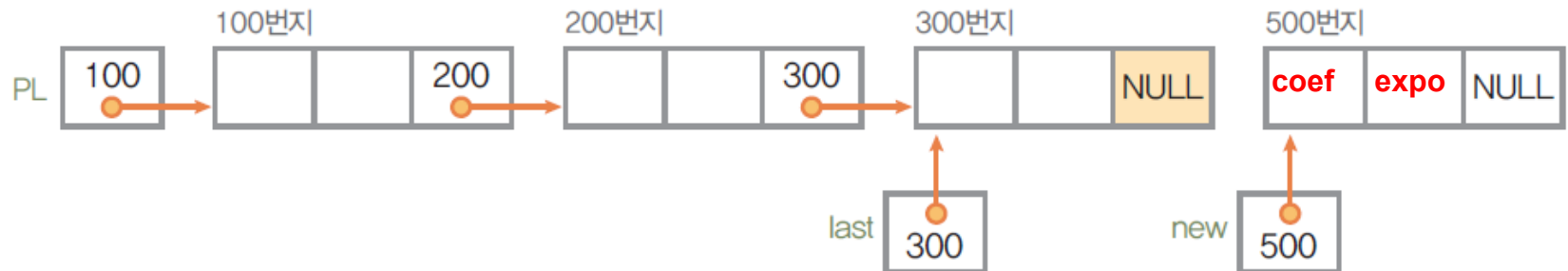
5. 연결 리스트의 응용 및 구현 : 항 삽입

② 공백 리스트가 아닌 경우

1) 초기상태

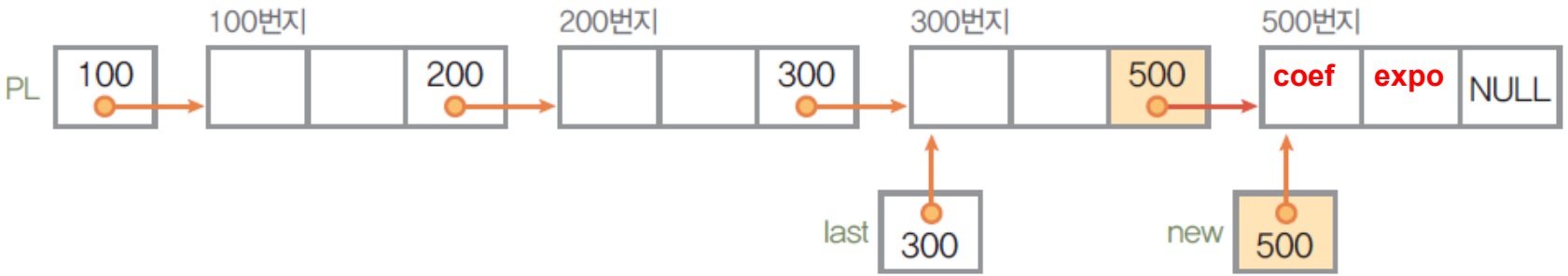


2) 노드를 만들고 노드 값을 저장

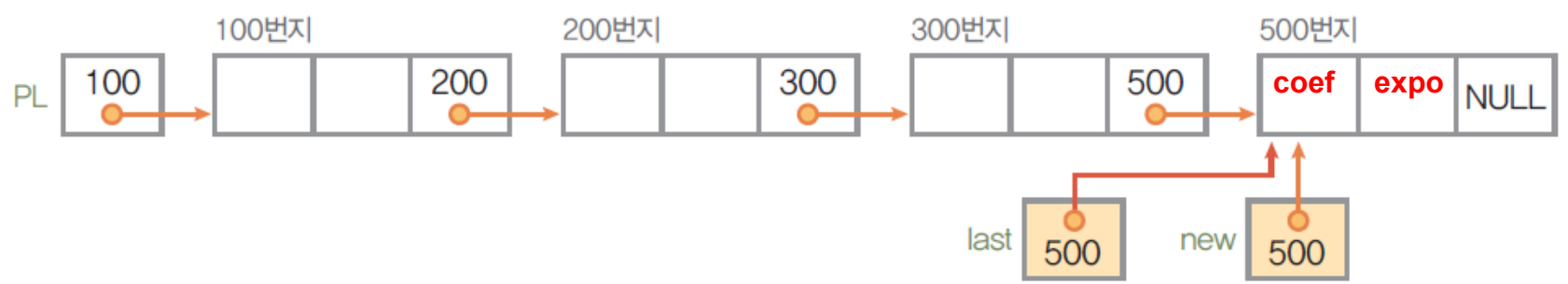


5. 연결 리스트의 응용 및 구현 : 항 삽입

2- a last.link ← new;

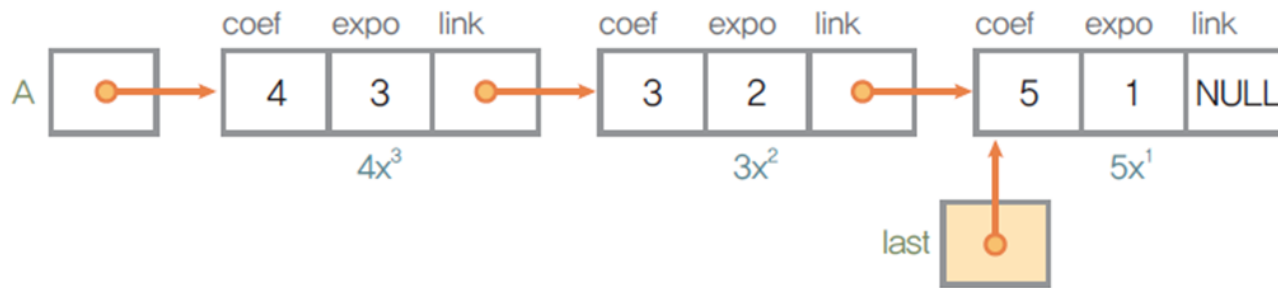


2- b last ← new;

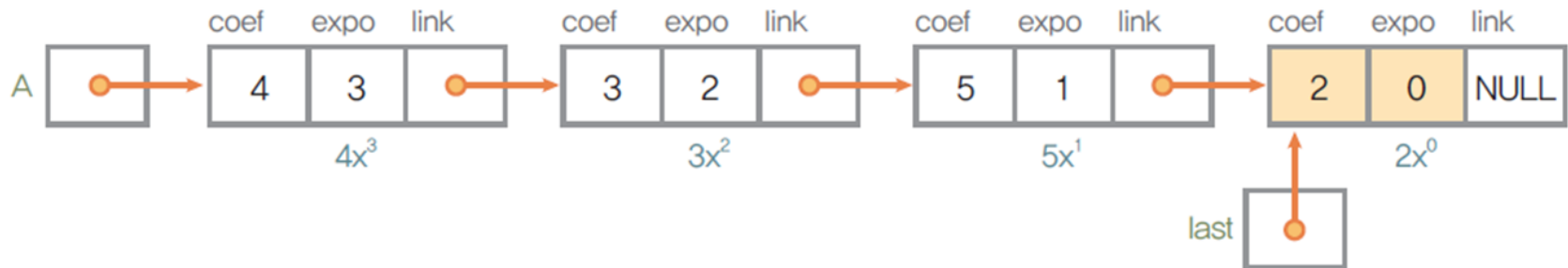


5. 연결 리스트의 응용 및 구현 : 항 삽입

- appendTerm() 함수를 이용해 다항식 리스트에 항을 추가한 예
 - 다항식 $A(x) = 4x^3 + 3x^2 + 5x$ 에 $2x^0$ 을 추가 하는 예시



(a) appendTerm(A, 2, 0, last) 실행 전



(b) appendTerm(A, 2, 0, last) 실행 후

그림 4-38 appendTerm() 함수를 이용해 다항식 리스트에 항을 추가한 예

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

■ 다항식끼리의 덧셈 연산과 알고리즘

- 덧셈 $A(x)+B(x)=C(x)$ 를 단순 연결 리스트 자료구조를 사용하여 연산
 - 다항식 $A(x)$ 와 $B(x)$, $C(x)$ 의 항을 지시하기 위해서 세 개의 포인터를 사용
 - 포인터 p : 다항식 $A(x)$ 에서 비교할 항을 지시
 - 포인터 q : 다항식 $B(x)$ 에서 비교할 항을 지시
 - 포인터 r : 덧셈연산 결과 만들어지는 다항식 $C(x)$ 의 항을 지시

$$A(x) = 4x^3 + 3x^2 + 5x$$

포인터 p

$$B(x) = 3x^4 + x^3 + 2x + 1$$

포인터 q

$$C(x) = 3x^4 + 5x^3 + 3x^2 + 7x + 1$$

포인터 r

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

1. $p.\text{expo} < q.\text{expo}$: 다항식 $A(x)$ 항의 지수가 작은 경우

- 두 다항식의 지수가 다르면 계수를 더할 수 없고, 지수가 높은 항부터 나열하는 다항식 표현 규칙에 따라서 포인터 q 가 가리키는 다항식 $B(x)$ 항을 $C(x)$ 항으로 복사

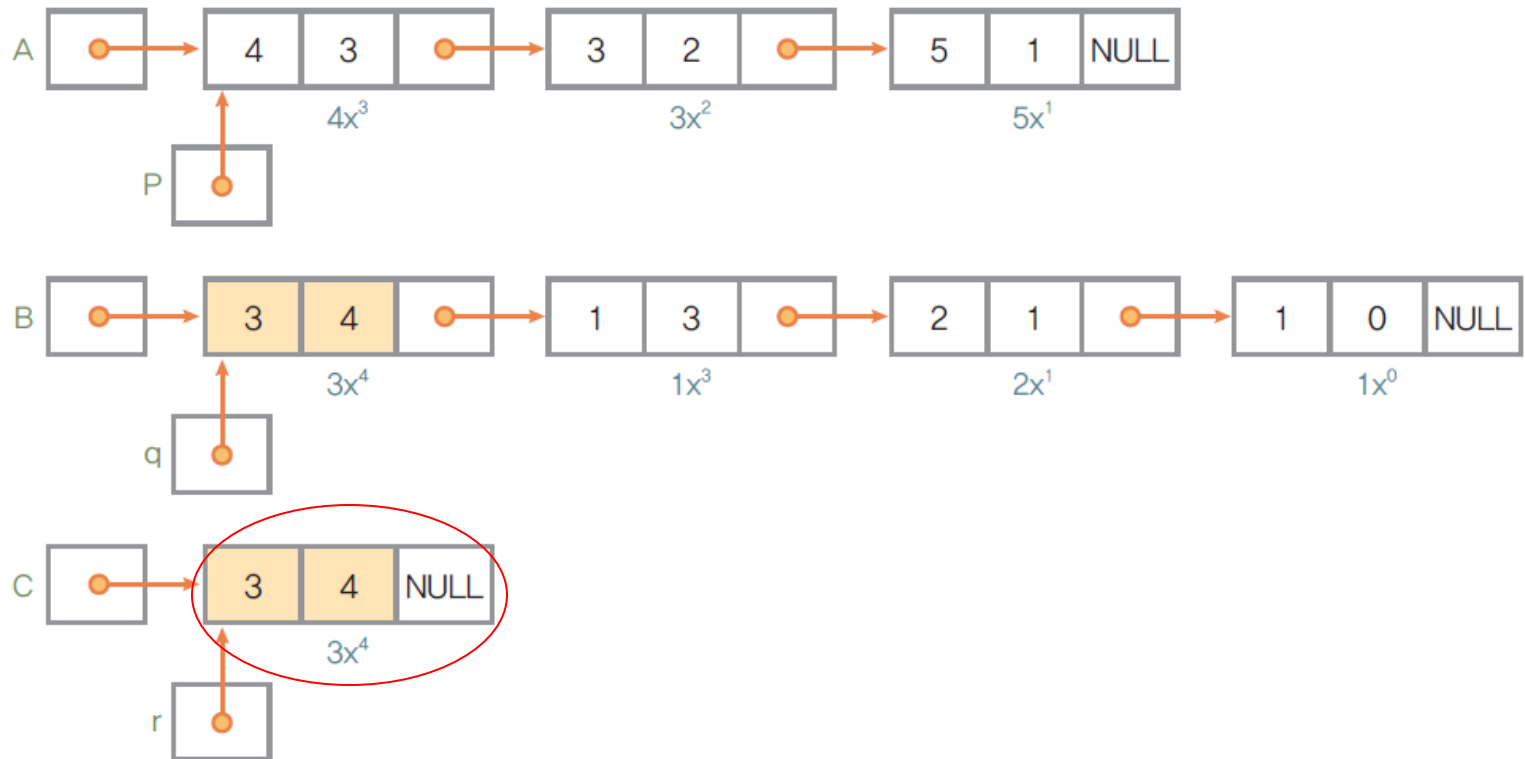


그림 4-39 q 의 지수가 더 큰 경우의 연산

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

1. $p.\text{expo} < q.\text{expo}$: 다항식 $A(x)$ 항의 지수가 작은 경우
 - q 가 가리키는 항에 대한 처리가 끝나면 q 를 다음 노드로 이동



5. 연결 리스트의 응용 및 구현 : 덧셈 연산

2. $p.\text{expo} = q.\text{expo}$: 두 다항식 항의 지수가 같은 경우

- 두 다항식의 지수가 같으면 지수가 같은 항의 계수를 더해 $C(x)$ 항을 만들면 되므로 $p.\text{coef}$ 와 $q.\text{coef}$ 를 더해 $C(x)$ 항인 $r.\text{coef}$ 에 저장하고, 지수는 $p.\text{expo}$ (또는 $q.\text{expo}$)를 $r.\text{expo}$ 에 저장

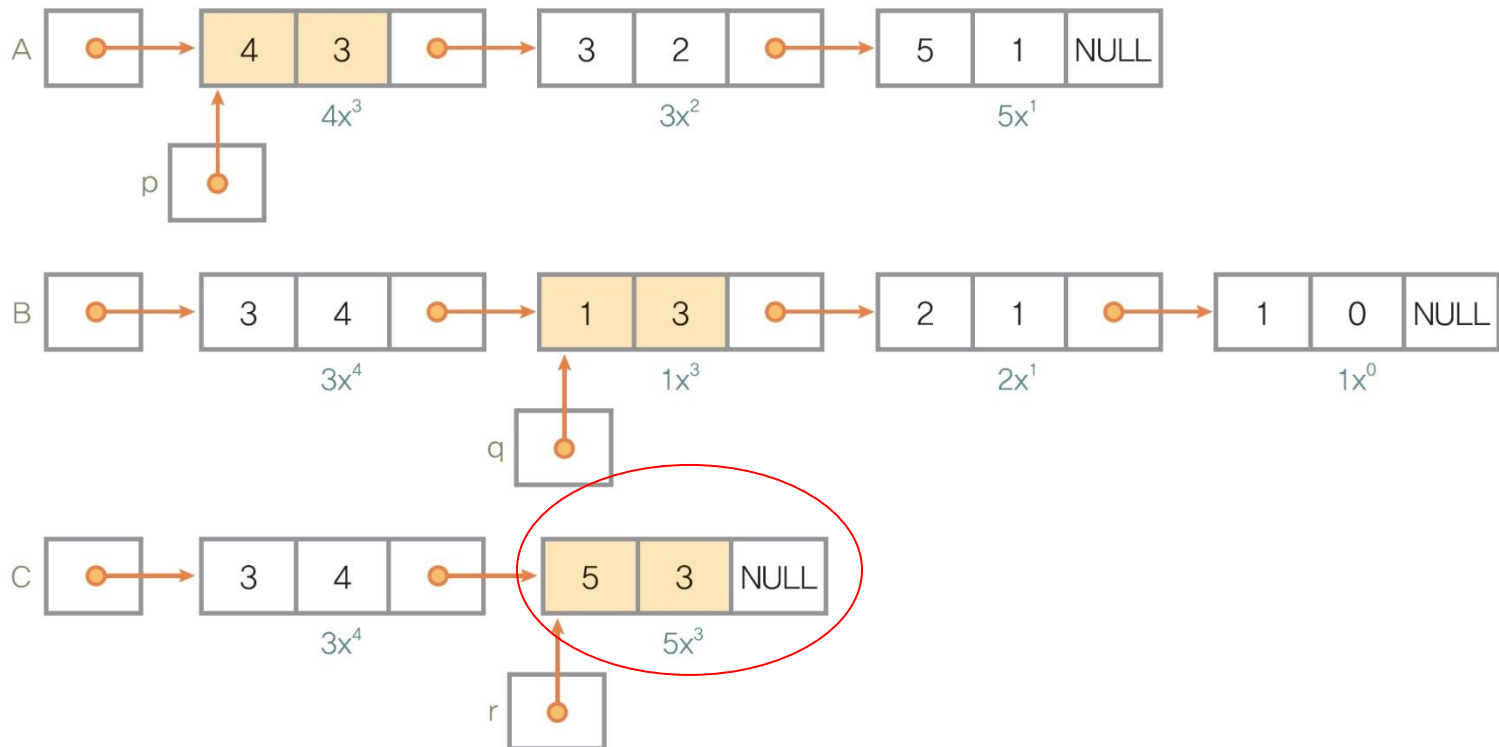
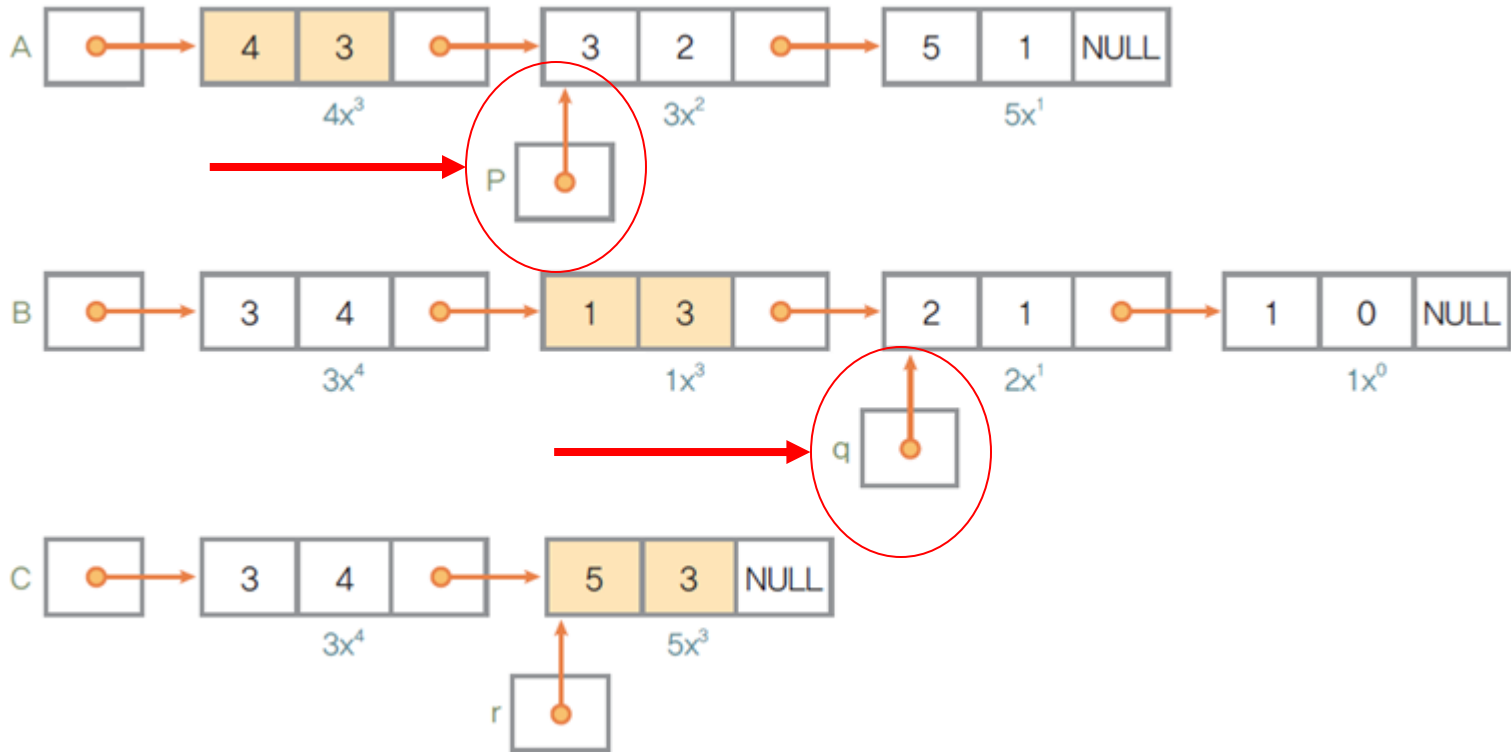


그림 4-40 지수가 같은 경우의 연산

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

2. $p.\text{expo} = q.\text{expo}$: 두 다항식 항의 지수가 같은 경우

- 다음 항을 비교하기 위해 포인터 p와 q를 각각 다음 노드로 이동



3. $p.expo > q.expo$: 다항식 $A(x)$ 항의 지수가 큰 경우
- p 가 가리키는 다항식 $A(x)$ 항의 지수가 더 크면 포인터 p 가 가리키는 항을 $C(x)$ 항으로 복사

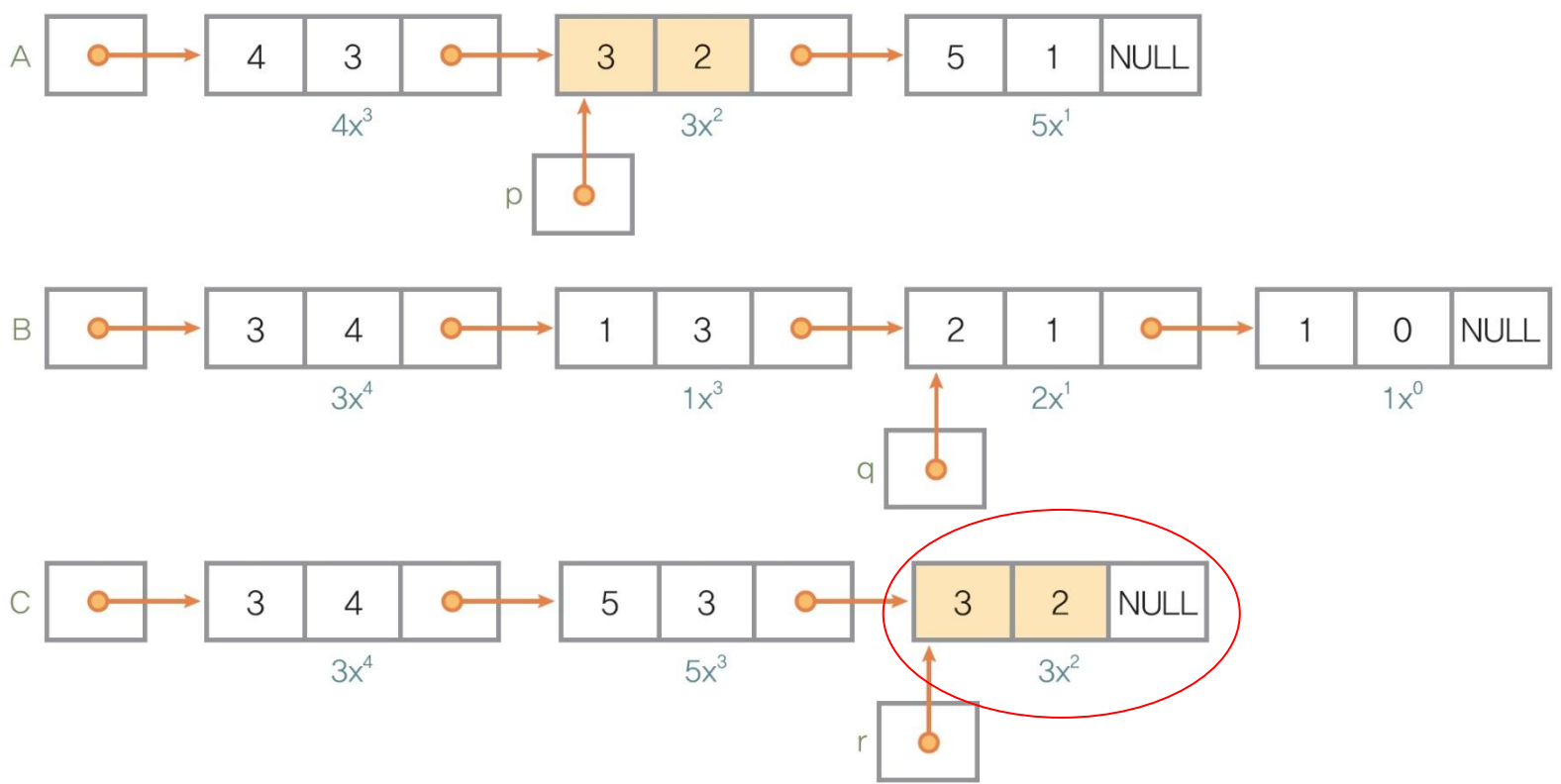
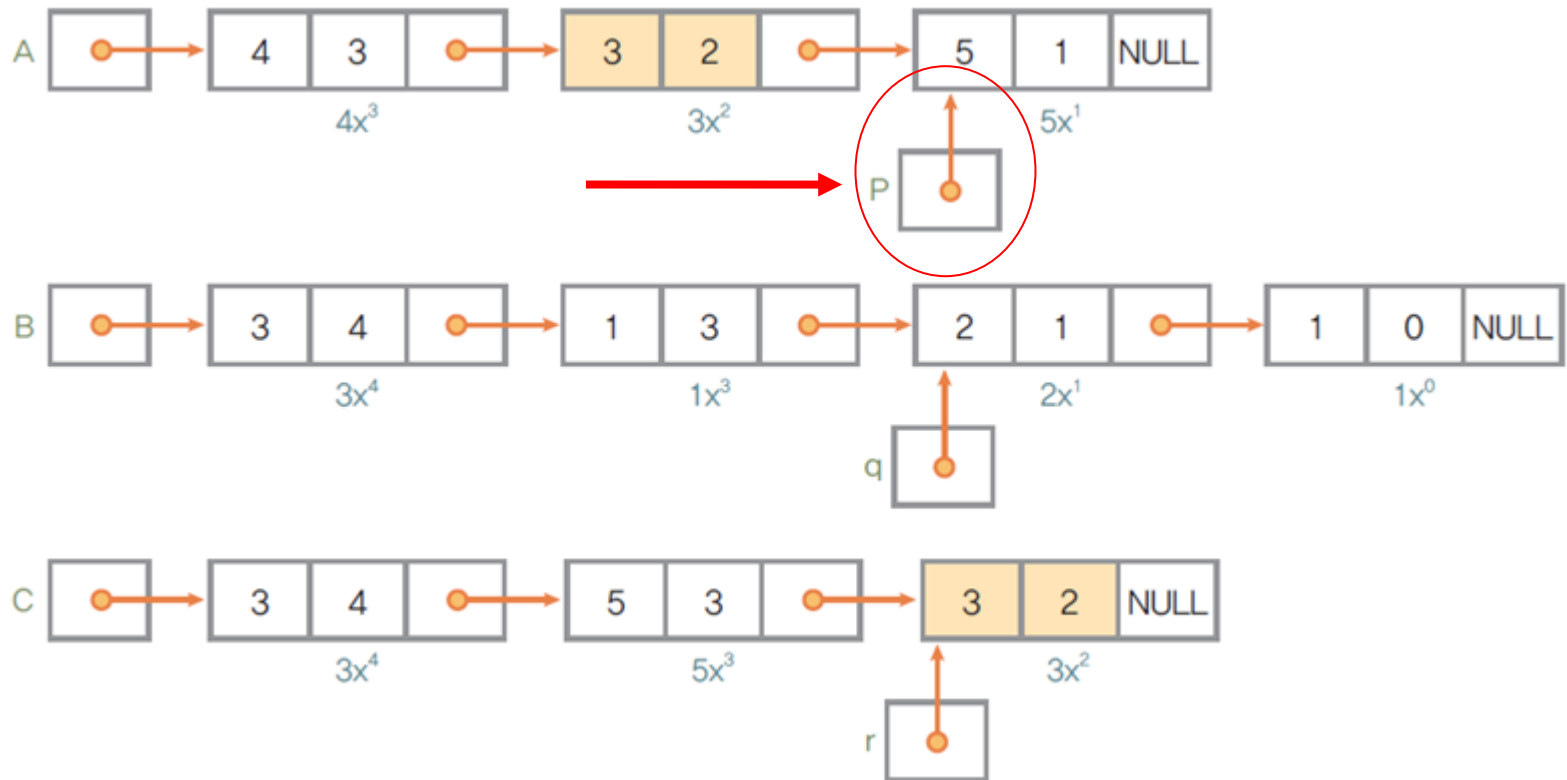


그림 4-41 p 의 지수가 더 큰 경우의 연산

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

3. $p.\text{expo} > q.\text{expo}$: 다항식 $A(x)$ 항의 지수가 큰 경우

- 포인터 p 가 가리키는 항에 대한 처리가 끝났으므로 p 를 다음 노드로 이동



5. 연결 리스트의 응용 및 구현 : 덧셈 연산

- 두 다항식의 덧셈 연산 알고리즘으로 정리

알고리즘 4-12 단순 연결 리스트를 이용한 다항식 덧셈 연산

```
// 단순 연결 리스트로 표현된 다항식 A와 B를 더해 새로운 다항식 C를 반환
addPoly(A, B)
{
    ① p ← A;
    q ← B;
    C ← NULL; // 결과 다항식
    r ← NULL; // 결과 다항식의 마지막 노드

    while (p ≠ NULL and q ≠ NULL) do { // 다항식 A와 B가 모두 항이 남아 있는 동안 반복
        case {
            p.expo == q.expo :
                sum ← p.coef + q.coef
                ②-a if (sum ≠ 0) then appendTerm(C, sum, p.expo, r);
                p ← p.link;
                q ← q.link;
            ② p.expo < q.expo :
                ②-b appendTerm(C, q.coef, q.expo, r);
                q ← q.link;
            ②-c else : // p.expo > q.expo인 경우
                appendTerm(C, p.coef, p.expo, r);
                p ← p.link;
        }
    }

    ③ while (p ≠ NULL) do { // A의 나머지 항들 복사
        appendTerm(C, p.coef, p.expo, r);
        p ← p.link;
    }

    ④ while (q ≠ NULL) do { // B의 나머지 항들 복사
        appendTerm(C, q.coef, q.expo, r);
        q ← q.link;
    }

    ⑤ return C;
}
end addPoly()
```

- 다항식 $A(x) = 4x^3 + 3x^2 + 5x$ 와 $B(x) = 3x^4 + x^3 + 2x + 1$ 을 단계별 과정 ①의 초기 상태

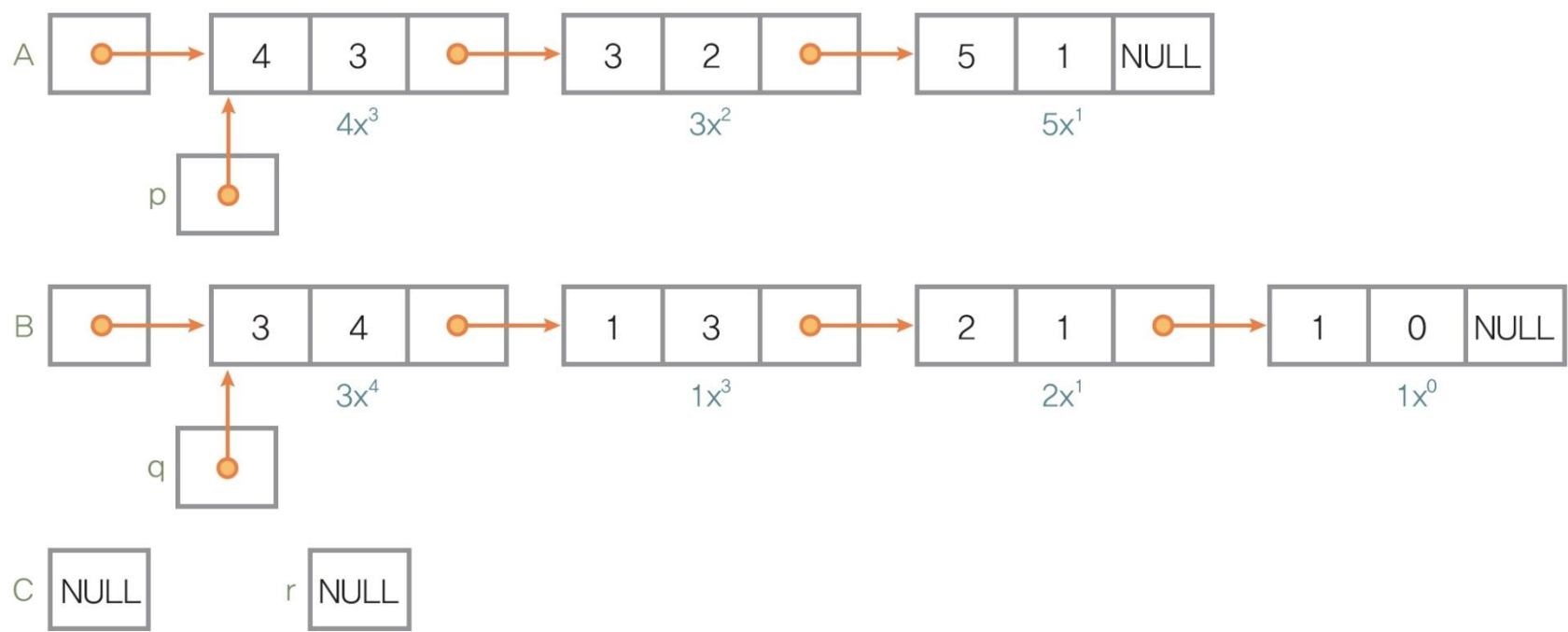


그림 4-42 초기 상태

- ②에서 다항식 A와 B에 처리 항 있는 동안, 항의 지수 비교 처리 작업 반복
- ②- b 수행

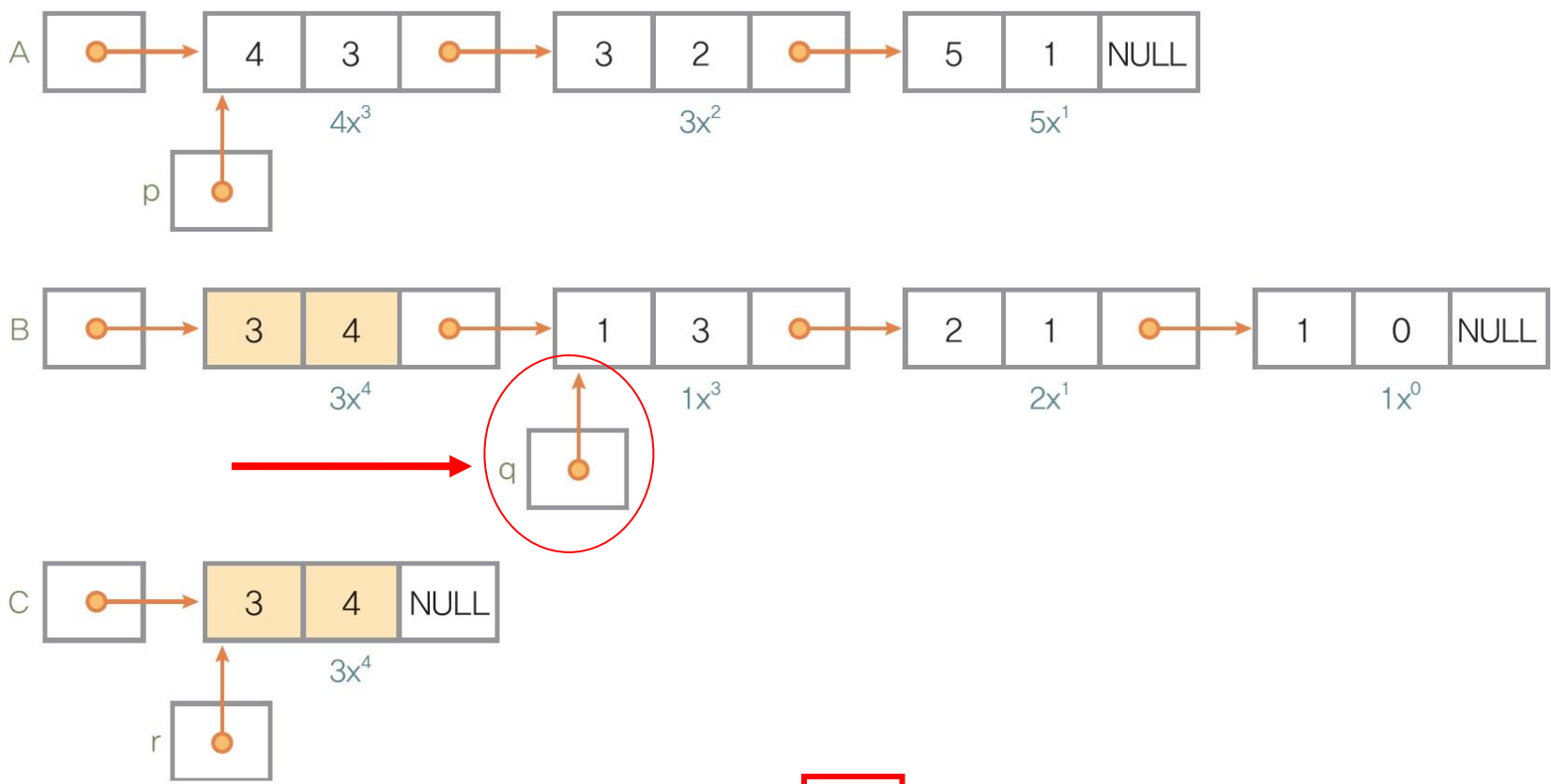


그림 4-43 다항식 A의 4x³과 다항식 B의 3x⁴을 처리한 결과

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

2-a 수행

- $p.\text{expo} = q.\text{expo}$ 이므로 두 노드의 coef 필드의 값을 더하고, expo 필드의 값을 복사한 노드($5x^3$ 항)를 리스트 C에 추가
- 포인터 p와 q는 다음 노드로 이동

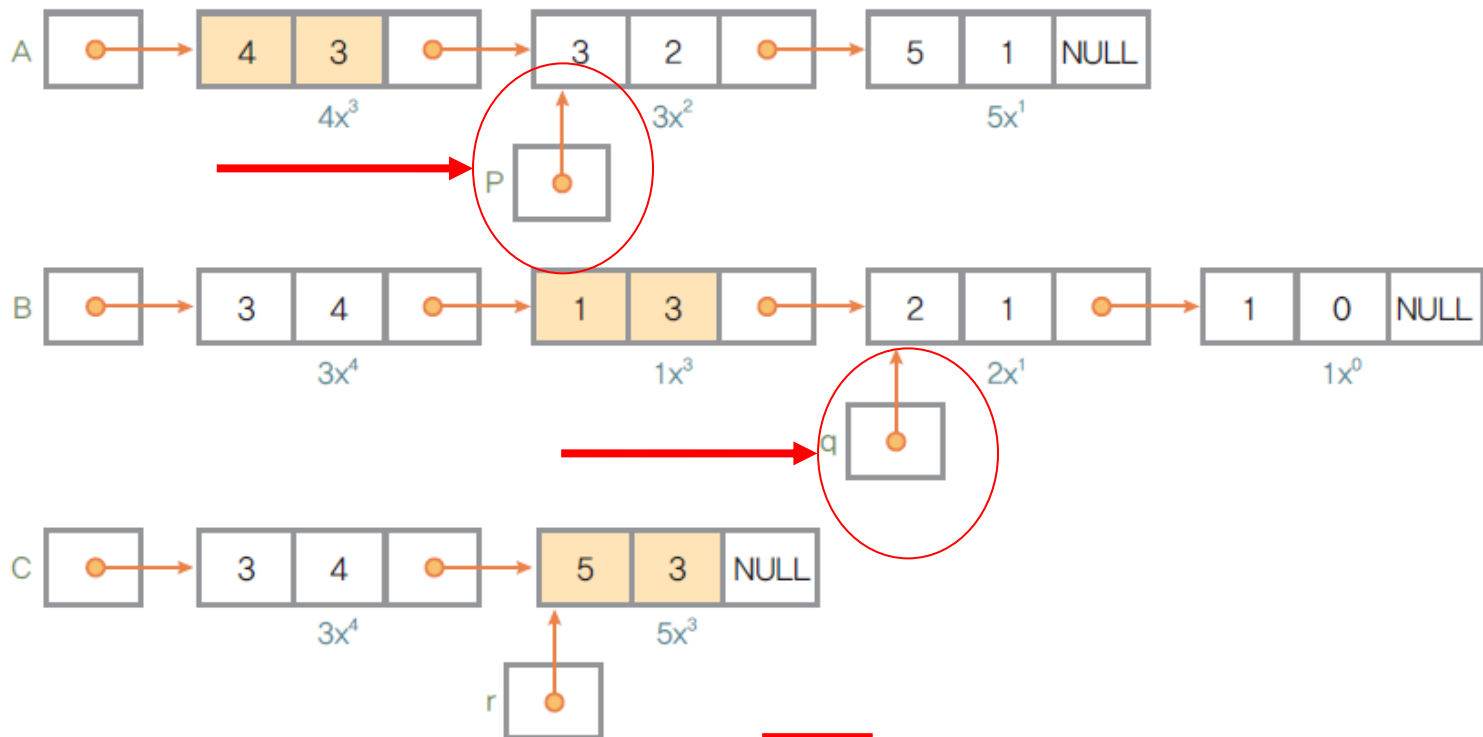


그림 4-44 다항식 A의 $4x^3$ 과 다항식 B의 $1x^3$ 을 처리한 결과

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

2-c 수행

- $p.\text{expo} > q.\text{expo}$ 이므로 지수가 큰 p 노드를 리스트 C에 복사
- 포인터 p는 다음 노드로 이동

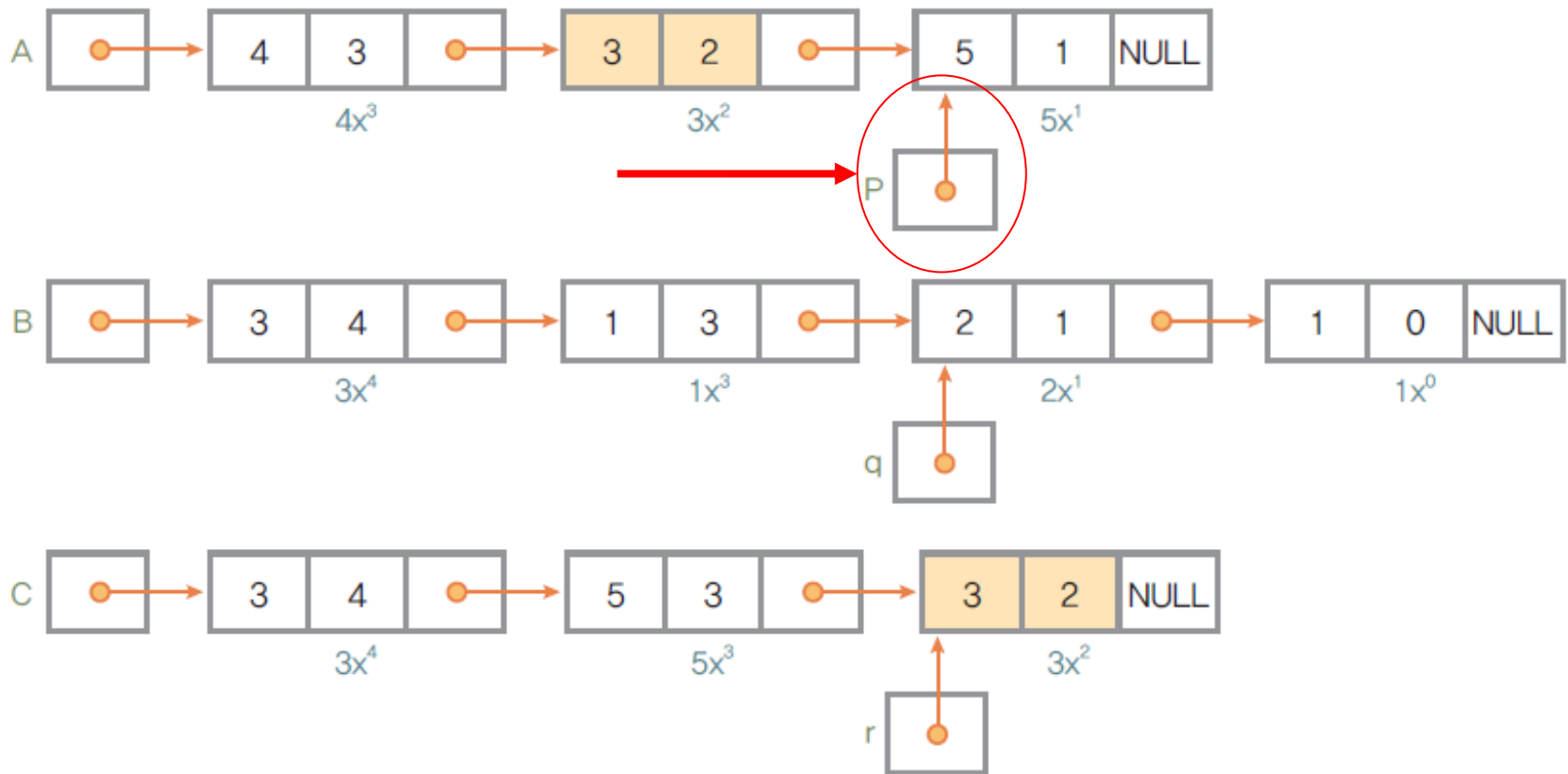


그림 4-45 다항식 A의 $3x^2$ 과 다항식 B의 $2x^1$ 를 처리한 결과

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

2-a 수행

- $p.\text{expo} = q.\text{expo}$ 이므로 두 노드의 coef 필드의 값을 더하고, expo 필드의 값을 복사한 노드($7x^1$ 항)를 리스트 C에 추가
- 포인터 p와 q는 다음 노드로 이동

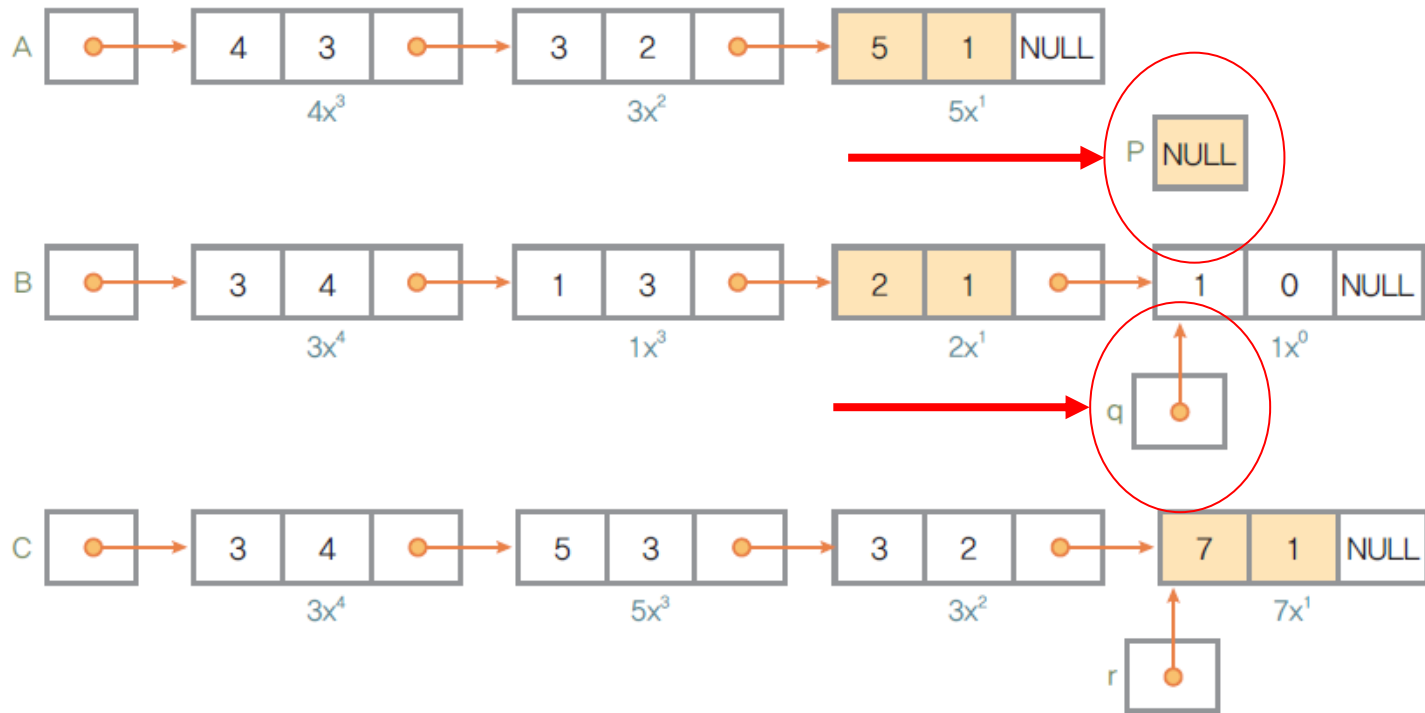


그림 4-46 다항식 A의 $5x^1$ 와 다항식 B의 $2x^1$ 를 처리한 결과

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

- B(x)의 남은 항에 대한 처리

- 다항식 A의 모든 항에 대한 처리가 끝나고 포인터 p가 NULL이 되었으므로 ②의 while 문 반복 끝나고 ④ 수행하여 포인터 q가 NULL이 될 때까지(while (q≠NULL)), q 노드를 리스트 C의 마지막 노드로 추가
- 마지막으로 ⑤에서 완성된 덧셈 결과 다항식 C 반환

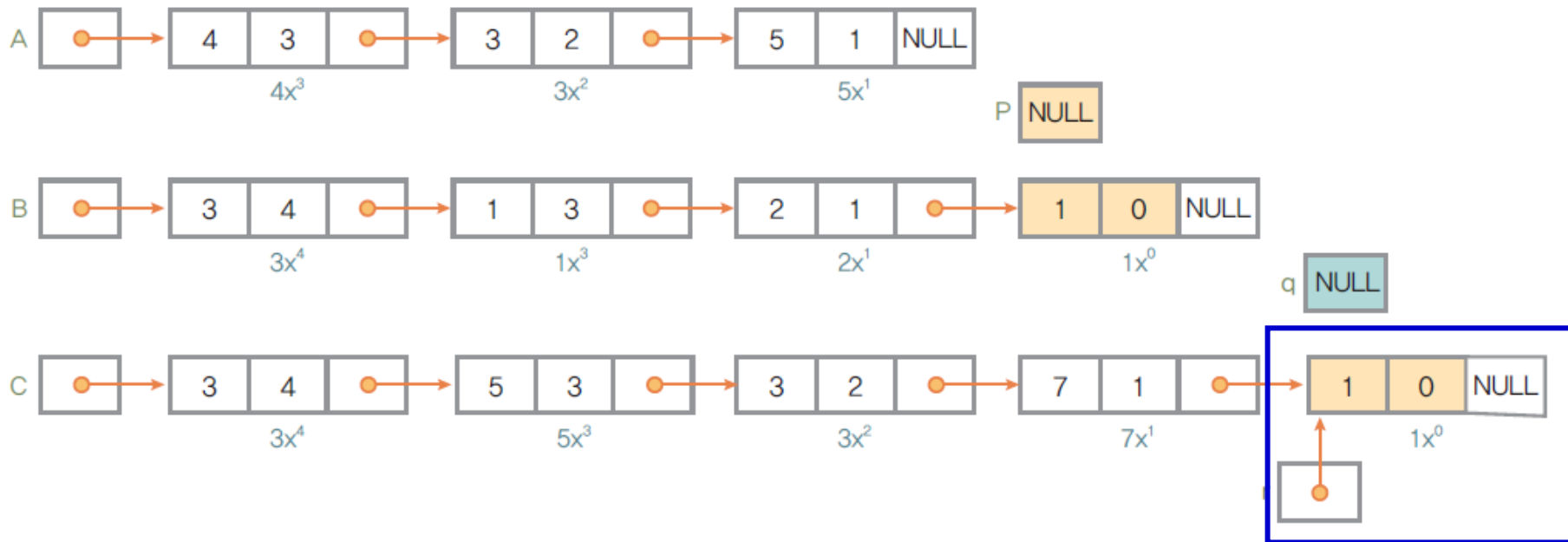
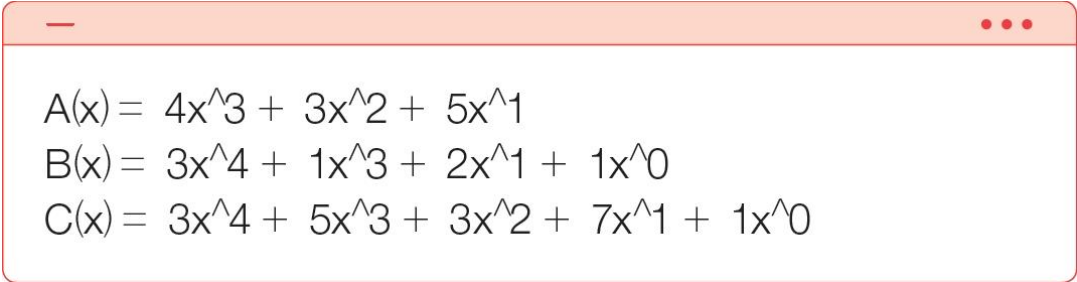


그림 4-47 다항식 $C(x) = 3x^4 + 5x^3 + 3x^2 + 7x + 1$

5. 연결 리스트의 응용 및 구현 : 덧셈 연산

- [예제 4-5] 연결 리스트를 이용한 다항식 프로그램 : [교재 222p](#)
- 실행 결과



```

A(x) = 4x^3 + 3x^2 + 5x^1
B(x) = 3x^4 + 1x^3 + 2x^1 + 1x^0
C(x) = 3x^4 + 5x^3 + 3x^2 + 7x^1 + 1x^0
    
```

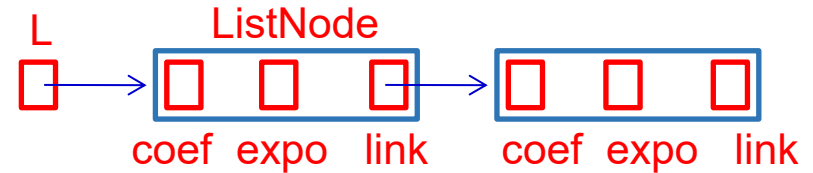
5. 연결 리스트의 응용 및 구현 : 덧셈 연산

■ LinkedPoly.h

주의) 예제 4-5. 교재와 소스 코드가 다름 !

```

LinkedPoly.h*
1  #pragma once
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  // 다항식 리스트의
6  float coef;
7  typedef struct ListNode {
8      float coef;
9      int expo;
10     struct ListNode* link;
11 } ListNode;
12
13 // 다항식 리스트의 head 노드를 구조체로 정의
14 //typedef struct ListHead {
15 //    ListNode* head;
16 //} ListHead;
17
18 ListNode* createLinkedList(void);
19 ListNode* appendTerm(ListNode* L, float coef, int expo);
20 ListNode* addPoly(ListNode* A, ListNode* B);
21 void printPoly(ListNode* L);
    
```



5. 연결 리스트의 응용 및 구현 : 덧셈 연산

■ LinkedPoly.c

```

LinkedPoly.c  (전역 범위)
1  #include "LinkedPoly.h"
2
3  // 공백 다항식 리스트를 생성하는 연산
4  ListNode* createLinkedList(void) {
5      ListNode* L;
6      L = NULL;
7      return L;
8  }
9
10 // 다항식 리스트에 마지막 노드를 추가하는 연산
11 ListNode* appendTerm(ListNode* L, float coef, int expo) {
12     ListNode* newNode;
13     ListNode* p;
14     newNode = (ListNode*)malloc(sizeof(ListNode));
15     newNode->coef = coef;
16     newNode->expo = expo;
17     newNode->link = NULL;
18
19     if (L == NULL) { // 다항식 리스트가 공백인 경우
20         L = newNode;
21     }
22     else { // 다항식 리스트가 공백이 아닌 경우
23         p = L;
24         while (p->link != NULL) {
25             p = p->link; // 리스트의 마지막 노드를 찾음
26         }
27         p->link = newNode; // 새 노드 연결
28     }
29     return L;
30 }

```

■ LinkedPoly.c (이어서)

```
31 // 두 다항식의 덧셈을 구하는 연산
32 ListNode* addPoly(ListNode* A, ListNode* B) {
33     ListNode* pA = A;
34     ListNode* pB = B;
35     ListNode* C = NULL;           // C 공백 다항식 초기화
36     float sum;
37
38     // 두 다항식에 노드가 있는 동안 반복 수행
39     while (pA && pB) {
40         // 다항식 A의 지수가 다항식 B의 지수와 같은 경우
41         if (pA->expo == pB->expo) {
42             sum = pA->coef + pB->coef;
43             C = appendTerm(C, sum, pA->expo);
44             pA = pA->link; pB = pB->link;
45         }
46         // 다항식 A의 지수가 다항식 B의 지수보다 큰 경우
47         else if (pA->expo > pB->expo) {
48             C = appendTerm(C, pA->coef, pA->expo);
49             pA = pA->link;
50         }
51         // 다항식 A의 지수가 다항식 B의 지수보다 작은 경우
52         else {
53             C = appendTerm(C, pB->coef, pB->expo);
54             pB = pB->link;
55         }
56     }
```

■ LinkedPoly.c (이어서)

```
57 // 다항식 A에 남아 있는 노드 복사
58 for (; pA != NULL; pA = pA->link)
59     C = appendTerm(C, pA->coef, pA->expo);
60
61 // 다항식 B에 남아 있는 노드 복사
62 for (; pB != NULL; pB = pB->link)
63     C = appendTerm(C, pB->coef, pB->expo);
64
65     return C;
66 }
67
68 // 다항식 리스트를 출력하는 연산
69 void printPoly(ListNode* L) {
70     ListNode* p = L;
71     for (; p; p = p->link) {
72         printf("%3.0fx^%d", p->coef, p->expo);
73         if (p->link != NULL) printf(" +");
74     }
75 }
```

■ Main 코드 ex4_5.c

```
ex4_5.c*
1  #include "LinkedPoly.h"
2
3  int main(void) {
4      ListNode *A, *B, *C;
5      // 공백 다항식 리스트 A, B, C 생성
6      A = createLinkedList();
7      B = createLinkedList();
8      C = createLinkedList();
9
10     A = appendTerm(A, 4, 3);    // 다항식 리스트 A에 4x3 노드 추가
11     A = appendTerm(A, 3, 2);    // 다항식 리스트 A에 3x2 노드 추가
12     A = appendTerm(A, 5, 1);    // 다항식 리스트 A에 5x1 노드 추가
13     printf("\n A(x) =");
14     printPoly(A);              // 다항식 리스트 A 출력
15
16     B = appendTerm(B, 3, 4);    // 다항식 리스트 B에 3x4 노드 추가
17     B = appendTerm(B, 1, 3);    // 다항식 리스트 B에 1x3 노드 추가
18     B = appendTerm(B, 2, 1);    // 다항식 리스트 B에 2x1 노드 추가
19     B = appendTerm(B, 1, 0);    // 다항식 리스트 B에 1x0 노드 추가
20     printf("\n B(x) =");
21     printPoly(B);              // 다항식 리스트 B 출력
22
23     C = addPoly(A, B);          // 다항식의 덧셈 연산 수행
24     printf("\n C(x) =");
25     printPoly(C);              // 다항식 리스트 C 출력
26
27     getchar(); return 0;
28 }
```

5. 연결 리스트의 응용 및 구현 : 곱셈 연산

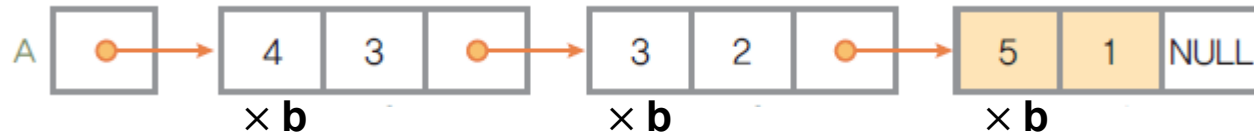
- 실습 : [예제 4-5] 연결리스트를 이용한 다항식에서
 - 다항식 A의 모든 항에 $b x^c$ 항을 곱하여 결과 다항식을 반환하는 함수 `ListNode* multTerm(ListNode *A, float b, int c)` 를 작성하시오.
 - `main( )` 함수에서 위의 두 함수 동작을 검증하시오.

– 1) 다항식 A 의 예

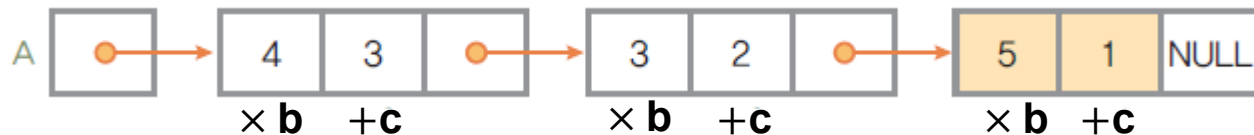


5. 연결 리스트의 응용 및 구현 : 곱셈 연산

- 2) 다항식 A 에 b 를 곱한 결과.



- 3) 다항식 A 에 bx^c 을 곱한 결과.



- multTerm() 의 동작후 변화된 연결리스트.

